머신러닝 · 앙상블 심화

고급 그래디언트 부스팅

XGBoost, LightGBM, CatBoost의 원리와 2026년 구현

원리는 손으로 따라가고, 코딩은 클로드 코드에게 맡긴다

StatQuest 예제로 세 엔진이 트리를 만드는 과정을 단계별로 재현하고,
구현은 클로드 코드에게 맡기는 프롬프트와 코딩 주의 사항(버전, 별칭)으로 마무리한다.

이 강의의 세 가지 메시지

세 엔진은 같은 부스팅 뼈대 위에서 서로 다른 병목을 공략한다

1

XGBoost는 2계 근사와 규제화로 트리를 직접 설계한다

잔차의 합과 개수로 유사도와 이득을 계산하고, λ와 γ로 트리 크기와 출력값을 조절한다.

2

LightGBM은 데이터를 덜 보고도 같은 분할을 찾는다

리프 중심 성장으로 손실을 빨리 줄이고, GOSS와 EFB로 계산할 행과 열을 줄인다.

3

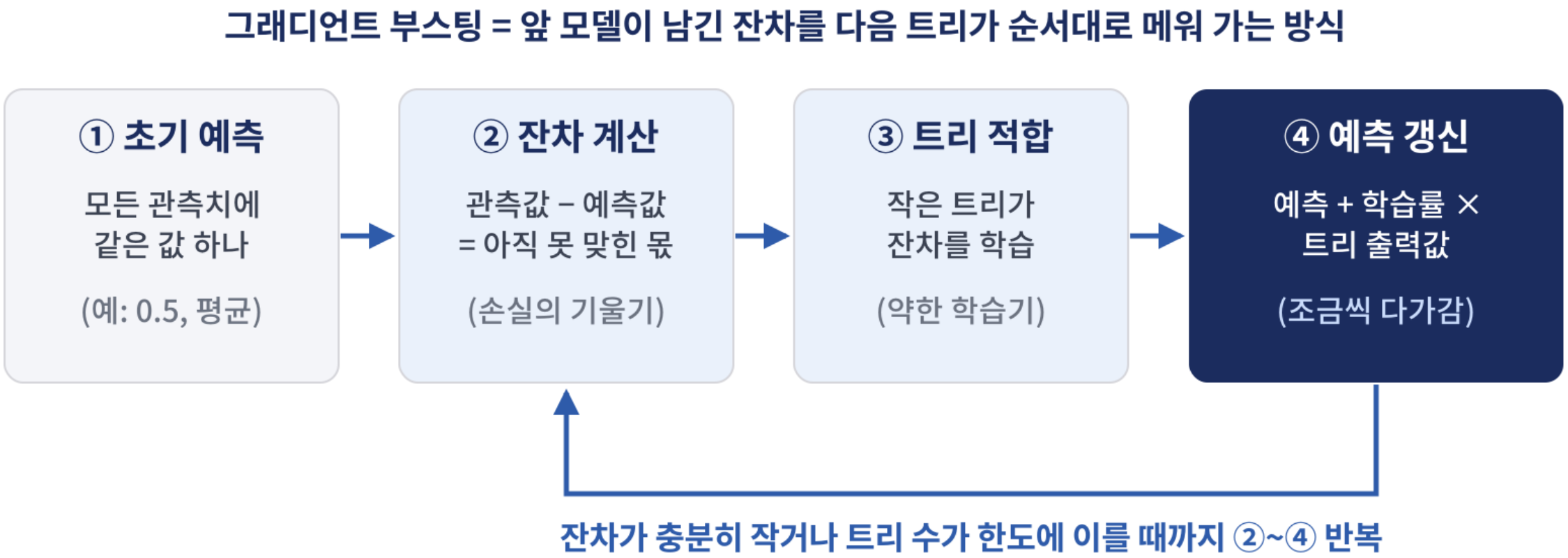
CatBoost는 순서를 지켜 타깃 누수를 막는다

Ordered Target Encoding과 Ordered Boosting으로 범주형 변수에서 생기는 과적합을 줄인다.

세 엔진의 정확도 차이는 작다. 데이터 구조와 속도 요구에 맞춰 고른다.

부스팅은 잔차를 순서대로 메워 가는 앙상블이다

그래디언트 부스팅의 네 단계 — 이 강의의 세 엔진이 모두 이 뼈대를 공유한다



세 엔진은 같은 뼈대에서 다른 병목을 공략한다

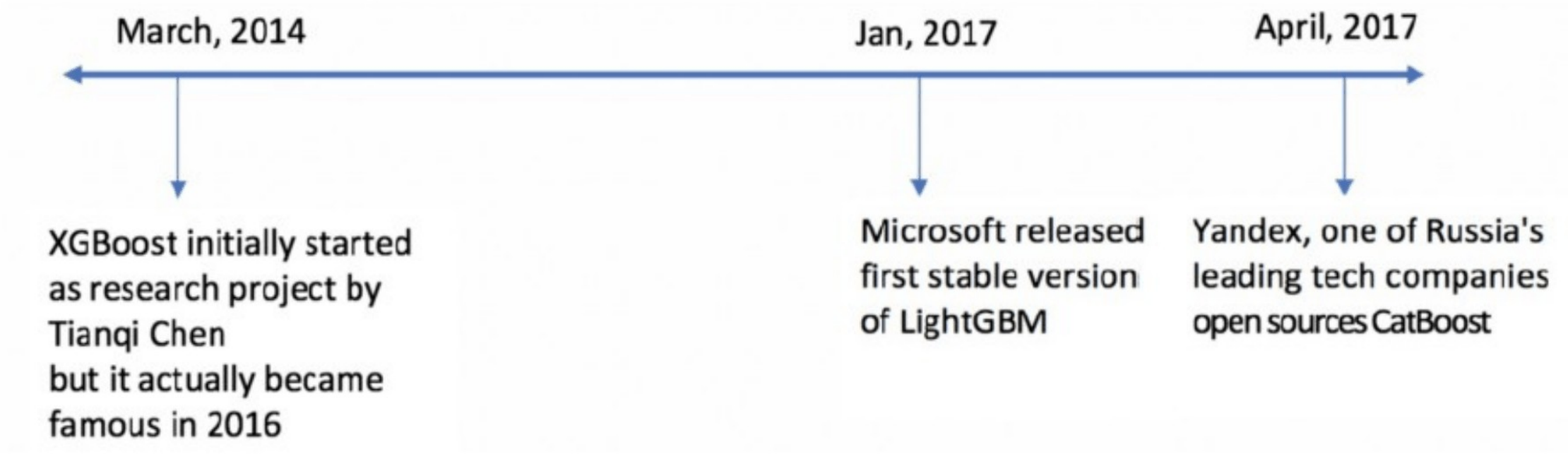
개발 배경과 핵심 아이디어 비교 — 버전은 2026년 11월 기준

구분	XGBoost	LightGBM	CatBoost
개발	Tianqi Chen (2014년 연구 시작)	Microsoft (2017년 1월 공개)	Yandex (2017년 4월 공개)
트리 성장	깊이 단위로 고르게 자란다	손실이 큰 리프부터 깊게 자란다	같은 깊이에서 같은 분할을 쓴다
핵심 기법	2계 근사로 분할 이득을 계산한다	큰 그래디언트만 남기고 변수를 묶는다	순서를 지켜 타깃 누수를 막는다
범주형 처리	enable_categorical을 켜면 쓴다	category 자료형을 자동 인식한다	기본으로 내장되어 가장 강하다
최신 버전	3.4.1 (2026년 8월)	4.7.0 (2026년 7월)	1.2.10 (2026년 2월)

지금은 세 모델을 섞는 블렌딩이 대회의 표준 조합이다.

XGBoost가 연 길을 두 엔진이 1년 사이에 뒤따랐다

발전 경과 — XGBoost는 2016년 Kaggle 우승으로 널리 알려졌다



01

PART 1 · XGBoost

XGBoost — 규제화된 2계 부스팅

목적함수를 보고, 회귀 트리 하나를 손으로 만든 뒤, 대용량을 위한 장치를 살핀다

XGBoost는 정확도와 시스템 최적화를 함께 잡았다

LightGBM 이전의 대표 GBDT로, 지금도 현업과 대회에서 널리 쓰인다

여섯 가지 강점

- 과적합을 막는 규제화를 목적함수에 넣었다.
- 결측값이 갈 방향을 학습한다.
- 트리를 병렬로 만든다.
- 캐시와 디스크를 고려해 대용량을 다룬다.
- 교차검증 기능을 내장했다.

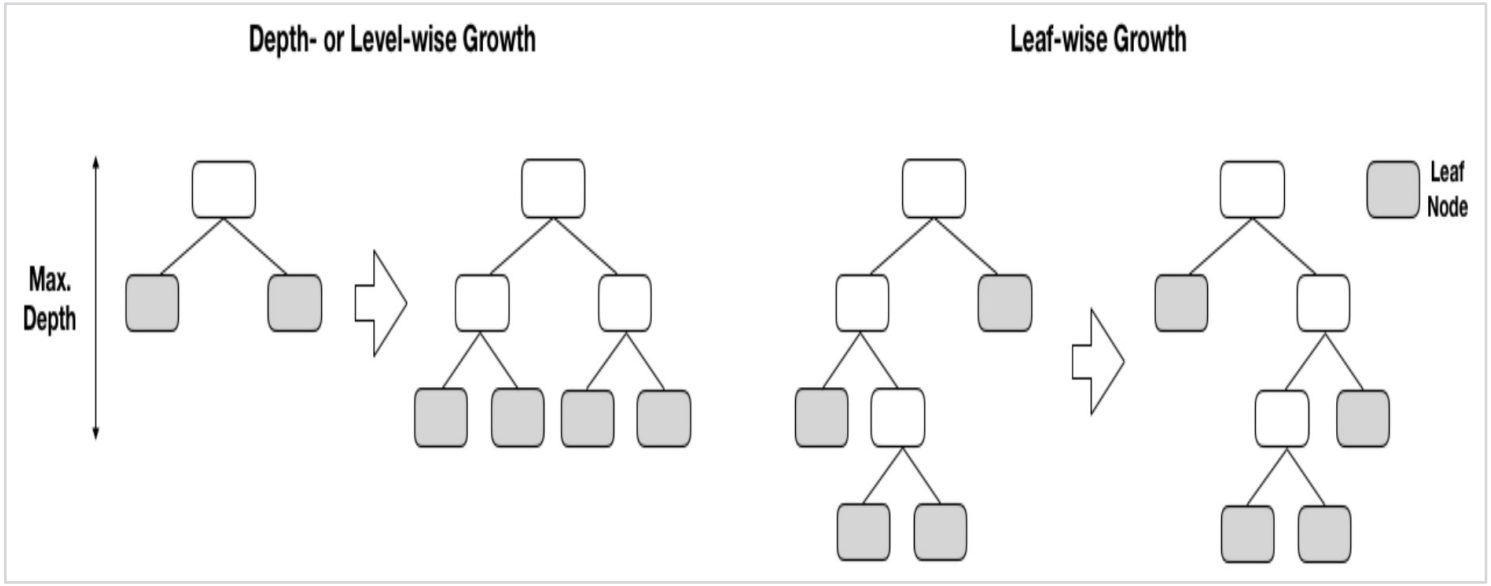


XGBoost 트리는 깊이 단위로 고르게 자란다

Depth-wise(level-wise) 성장 — 한쪽으로만 깊어지지 않는다

읽는 법

- 왼쪽은 같은 깊이의 노드를 모두 나눈 뒤 다음 깊이로 내려간다.
- 오른쪽은 LightGBM의 리프 중심 성장으로, PART 2에서 다시 본다.
- 깊이 단위 성장은 느리지만 과적합 위험이 낮다.



목적함수는 예측 손실에 트리 복잡도 벌점을 더한다

원본 강의자료의 규제화 목적함수 (Chen & Guestrin, 2016)

$$\mathcal{L}(\phi) = \underbrace{\sum_i l(\hat{y}_i, y_i)}_{\text{예측 손실}} + \underbrace{\sum_k \Omega(f_k)}_{\text{트리 복잡도 벌점}}$$

$$\Omega(f) = \underbrace{\gamma T}_{\text{리프 개수 벌점}} + \underbrace{\frac{1}{2}\lambda \|w\|^2}_{\text{리프 출력 크기 벌점}}$$

γ의 역할 (리프 개수)

- 리프가 하나 늘 때마다 γ만큼 벌점을 준다.

λ의 역할 (리프 출력)

- 리프 출력이 커질수록 벌점이 커진다.

분할 점수는 그래디언트와 헤시안의 합으로 구한다

2계 테일러 근사로 얻은 분할 점수 — 손실함수에서 분지점을 직접 구한다

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{(\sum_{i\in I_L} g_i)^2}{\sum_{i\in I_L} h_i+\lambda} + \frac{(\sum_{i\in I_R} g_i)^2}{\sum_{i\in I_R} h_i+\lambda} - \frac{(\sum_{i\in I} g_i)^2}{\sum_{i\in I} h_i+\lambda}\right] - \gamma$$

기호 읽기

- g는 손실의 1계 도함수(그래디언트), h는 2계 도함수(헤시안)다.
- 괄호 안은 왼쪽 노드, 오른쪽 노드, 분할 전 노드의 점수다.

의미

- 왼쪽과 오른쪽 점수의 합이 분할 전보다 γ 이상 커야 나눈다.
- 제곱오차 손실이면 h가 1이 되어, 뒤의 유사도 공식과 같아진다.

리프 출력값은 근사 손실을 최소로 만드는 값이다

원본의 '수학적 배경(Just Look!)' — 미분해 0으로 놓으면 출력값이 나온다

세 단계

- ① 리프 출력에 대해 근사 손실을 미분한다.
- ② 미분값을 0으로 놓는다.
- ③ 풀면 그래디언트 합을 헤시안 합과 λ의 합으로 나눈 값이 된다.

To find the O_{value} that minimizes... $\left[\sum_{i=1}^{n} L(y_i, p_i^0 + O_{value})\right] + \gamma + \frac{1}{2}\lambda O_{value}^2$

1) Take the derivative... $\frac{d}{dO_{value}}(g_1 + g_2 + g_3)O_{value} + \frac{1}{2}(h_1 + h_2 + h_3 + \lambda)O_{value}^2$

2) Set to zero... $(g_1 + g_2 + g_3) + (h_1 + h_2 + h_3 + \lambda)O_{value} = 0$

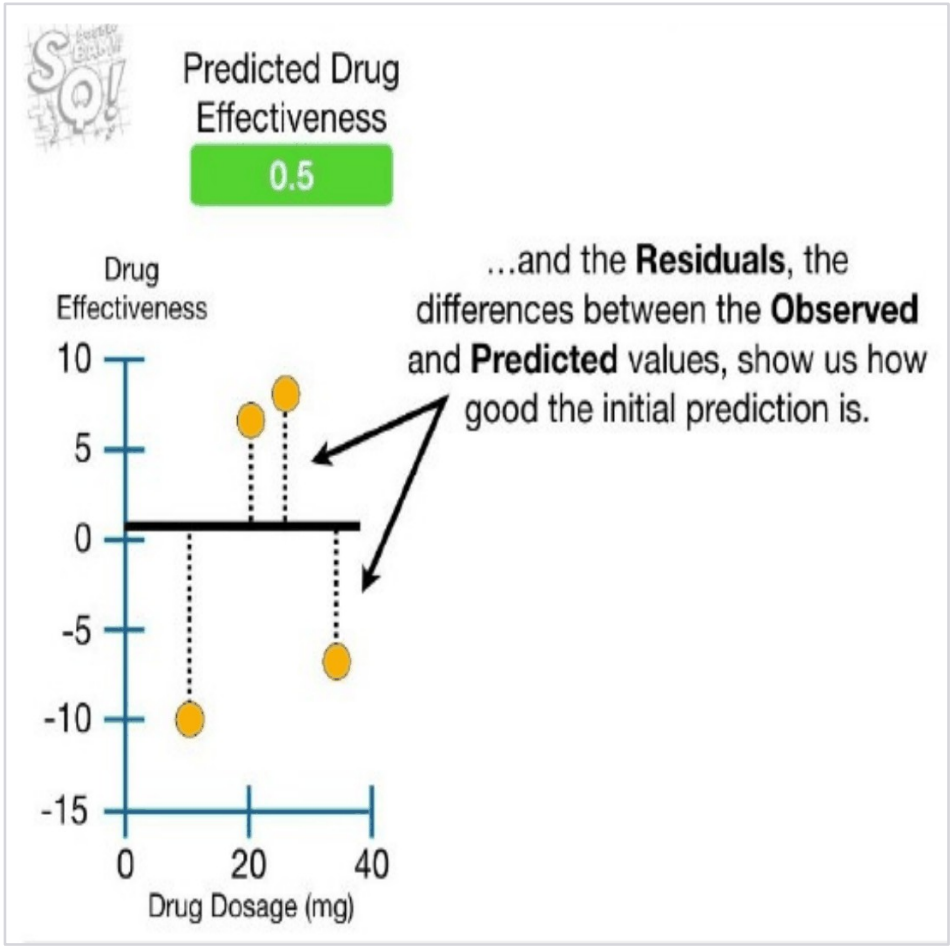
3) Solve for $f_k(x_i)$... $O_{value} = \frac{-(g_1 + g_2 + g_3)}{(h_1 + h_2 + h_3 + \lambda)}$

첫 예측값 0.5에서 잔차를 구하며 시작한다

StatQuest 예제 — 약물 용량으로 약효를 예측하는 관측치 4개

출발점

- XGBoost 회귀 트리는 부스팅, 규제화, 고유한 회귀 트리의 세 요소로 이뤄진다.
- 초기 예측은 기본값 0.5다.
- 잔차는 −10.5, 6.5, 7.5, −7.5다.

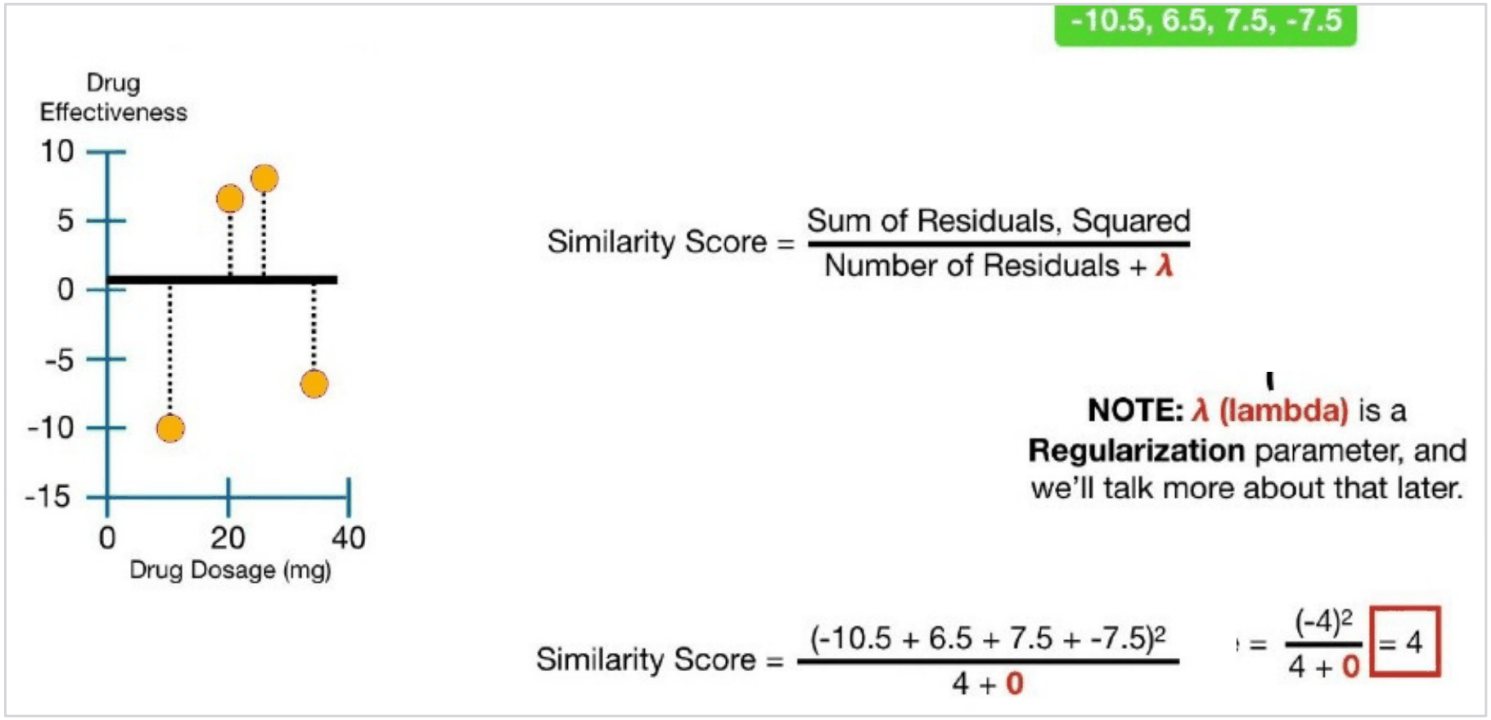


모든 잔차를 한 리프에 넣고 유사도를 계산한다

λ를 0으로 두고 뿌리 노드의 유사도를 구한다

읽는 법

- 네 잔차가 모두 뿌리 리프에 있다.
- 부호가 반대인 잔차가 서로 상쇄돼 유사도가 4로 작다.
- λ는 규제화 파라미터로, 효과는 뒤에서 본다.

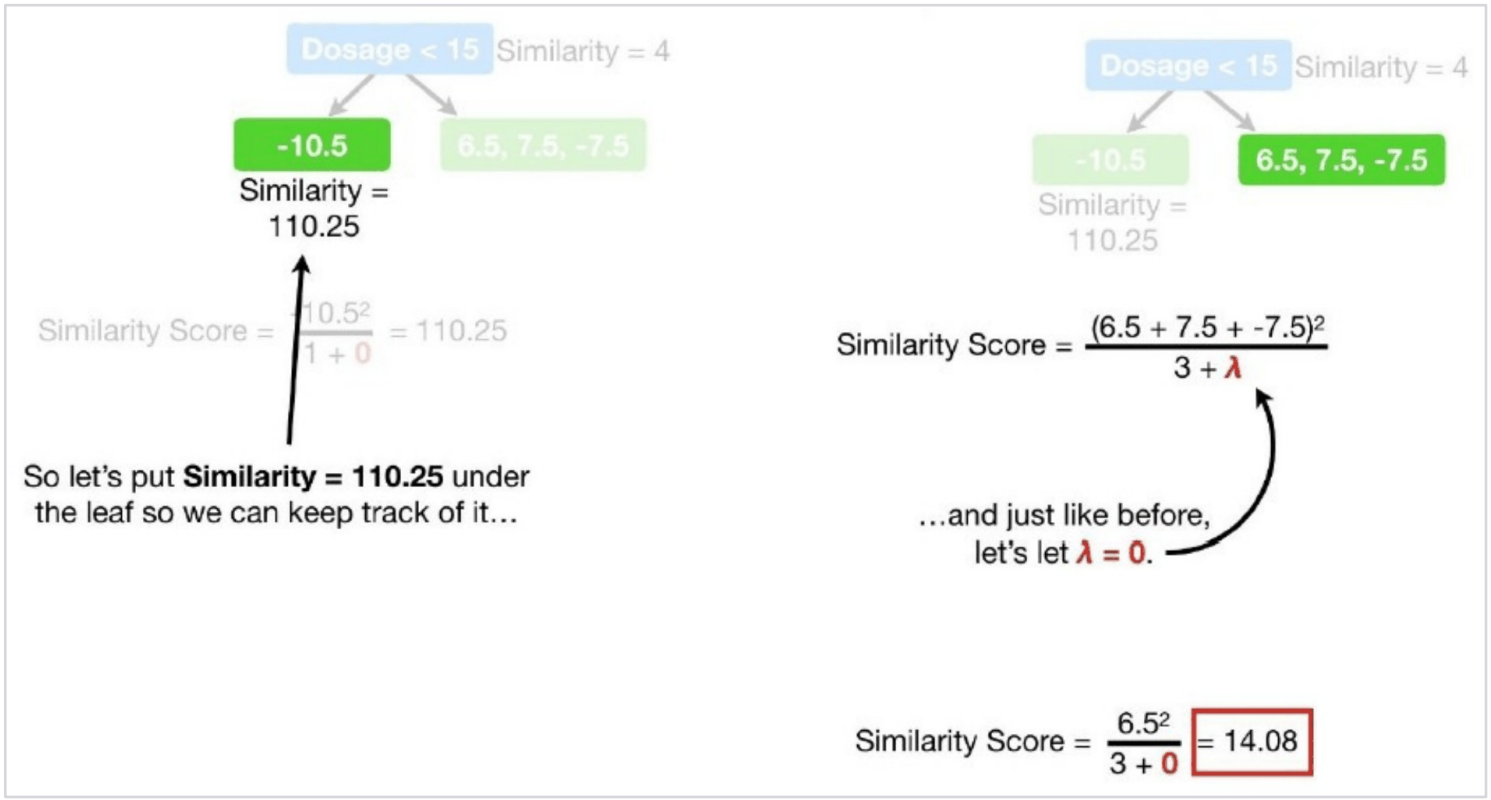


첫 분할 후보는 용량 15 미만이다

이웃한 두 관측치의 평균을 경계로 잡고 양쪽 유사도를 계산한다

읽는 법

- 왼쪽에는 잔차 −10.5 하나만 남아 유사도가 110.25로 커진다.
- 오른쪽은 6.5, 7.5, −7.5가 상쇄돼 14.08이다.
- 비슷한 잔차끼리 모일수록 유사도가 커진다.

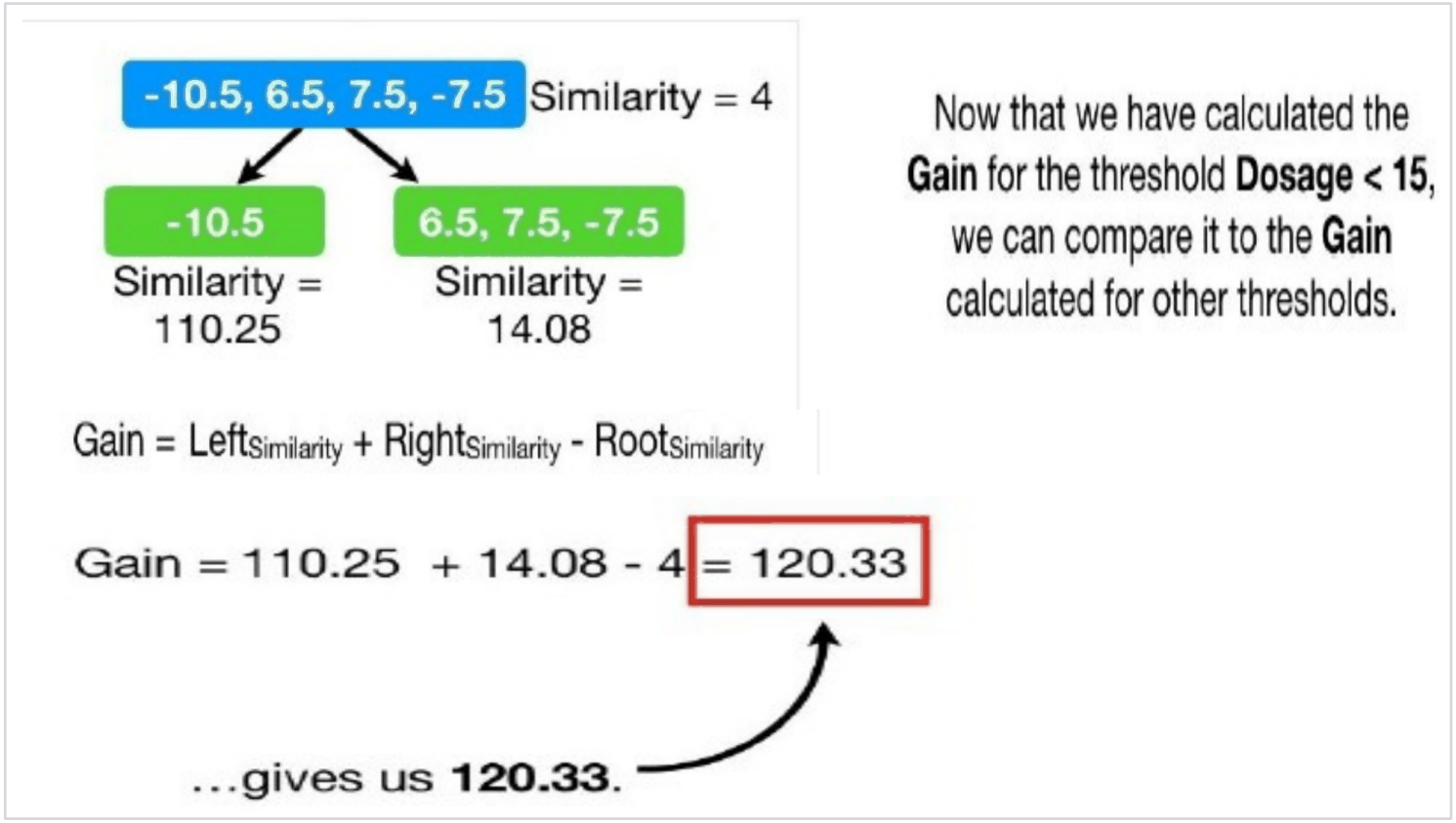


분할의 가치는 이득으로 잰다

용량 15 미만 분할의 이득은 110.25 + 14.08 − 4 = 120.33이다

읽는 법

- 이득은 왼쪽과 오른쪽 유사도를 더하고 뿌리 유사도를 뺀 값이다.
- 다른 경계값의 이득도 같은 방식으로 계산해 비교한다.

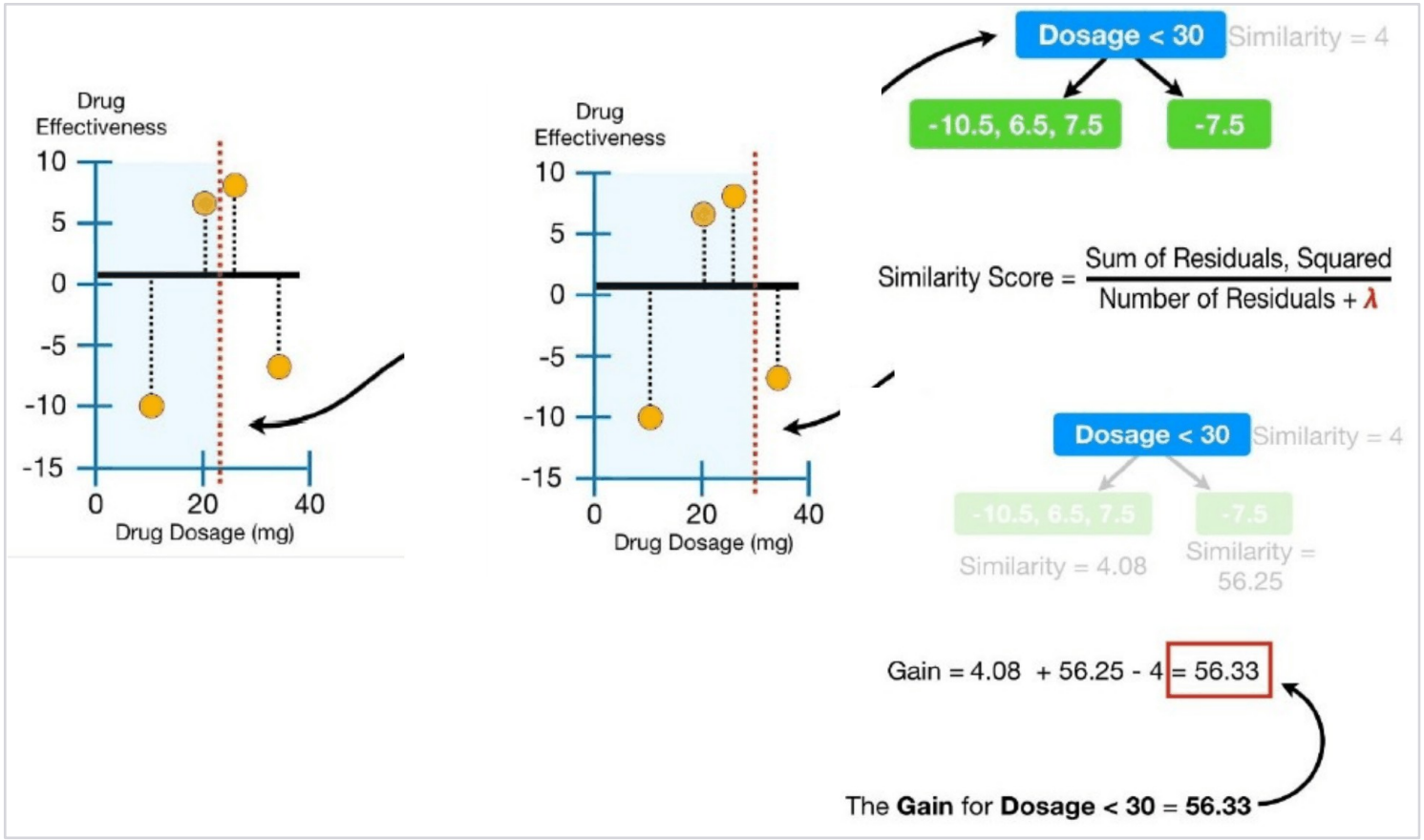


이득이 가장 큰 경계를 첫 분할로 고른다

용량 22.5 미만은 이득 4, 30 미만은 56.33으로 15 미만보다 작다

읽는 법

- 경계를 한 칸씩 옮기며 이득을 다시 계산한다.
- 세 후보 가운데 용량 15 미만의 이득(120.33)이 가장 크다.
- 모든 후보를 다 시험하는 이 방식을 탐욕 알고리즘이라 한다.

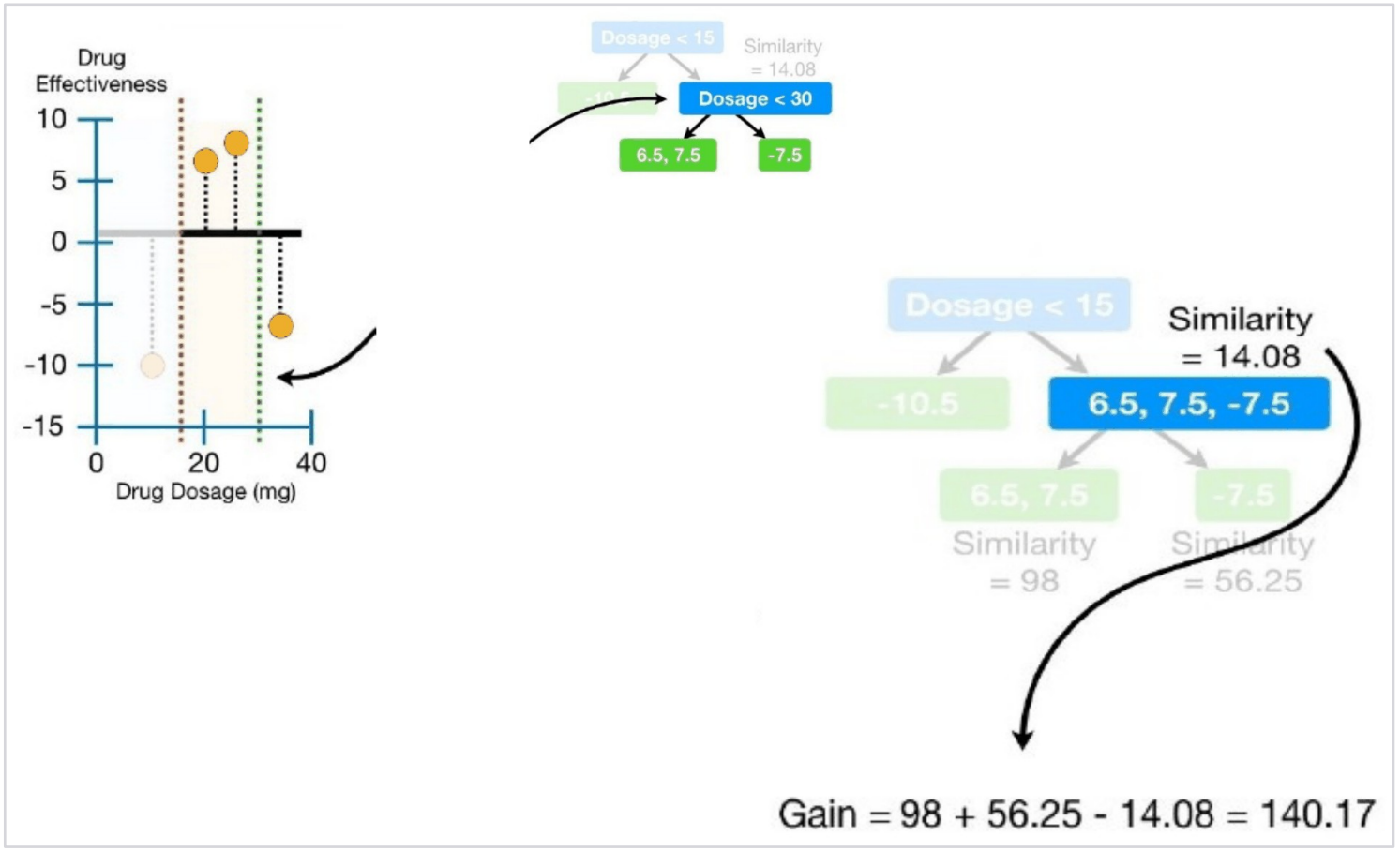


오른쪽 리프도 같은 방식으로 한 번 더 나눈다

왼쪽 리프는 잔차가 하나뿐이라 더 나눌 수 없다

읽는 법

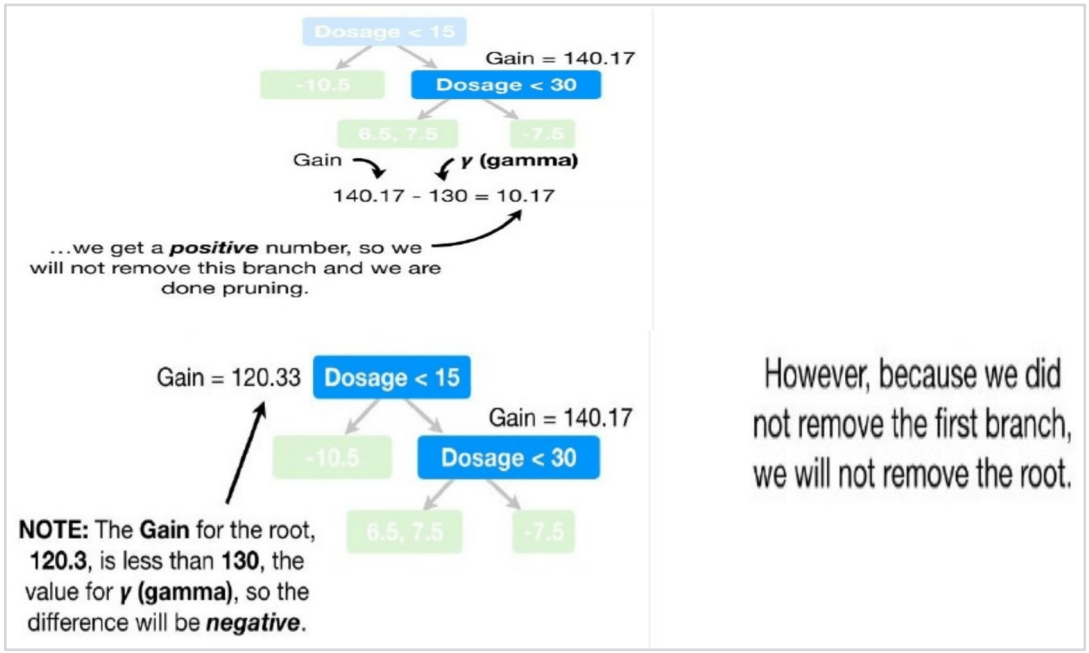
- 오른쪽 잔차 6.5, 7.5, −7.5를 두 경계로 나눠 본다.
- 용량 30 미만의 이득이 140.17로, 22.5 미만(28.17)보다 크다.
- 예제는 깊이를 2로 제한했다. 기본값은 6이다.



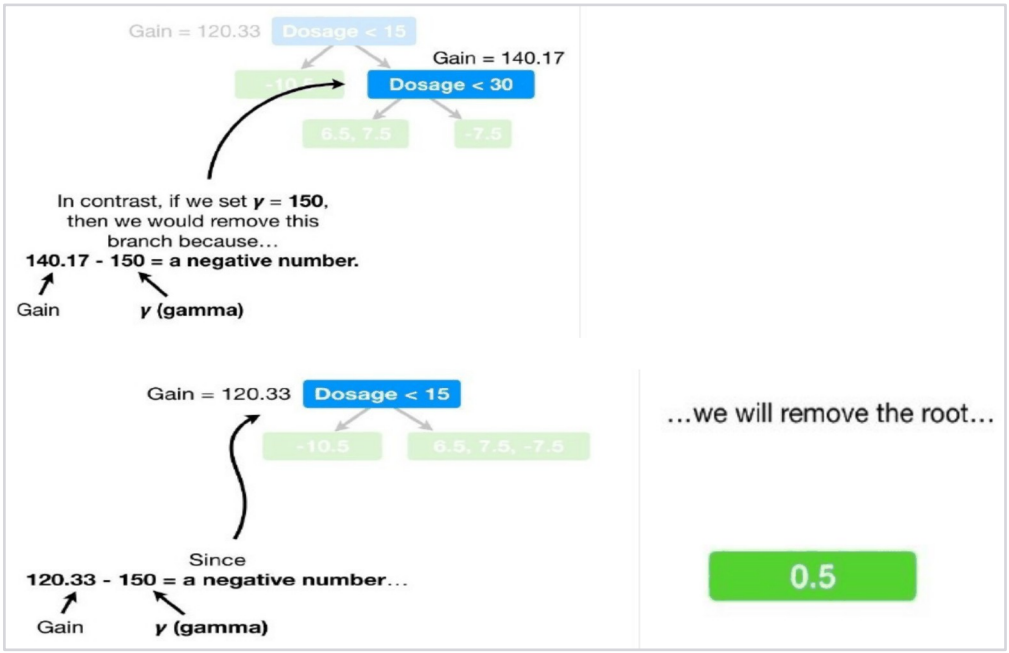
γ보다 이득이 작은 가지는 잘라 낸다

가지치기는 맨 아래 가지부터 '이득 − γ'의 부호로 판단한다

γ = 130이면 아래 가지의 이득 140.17이 더 커서 가지를 남긴다



γ = 150이면 아래 가지와 뿌리까지 모두 잘라 낸다



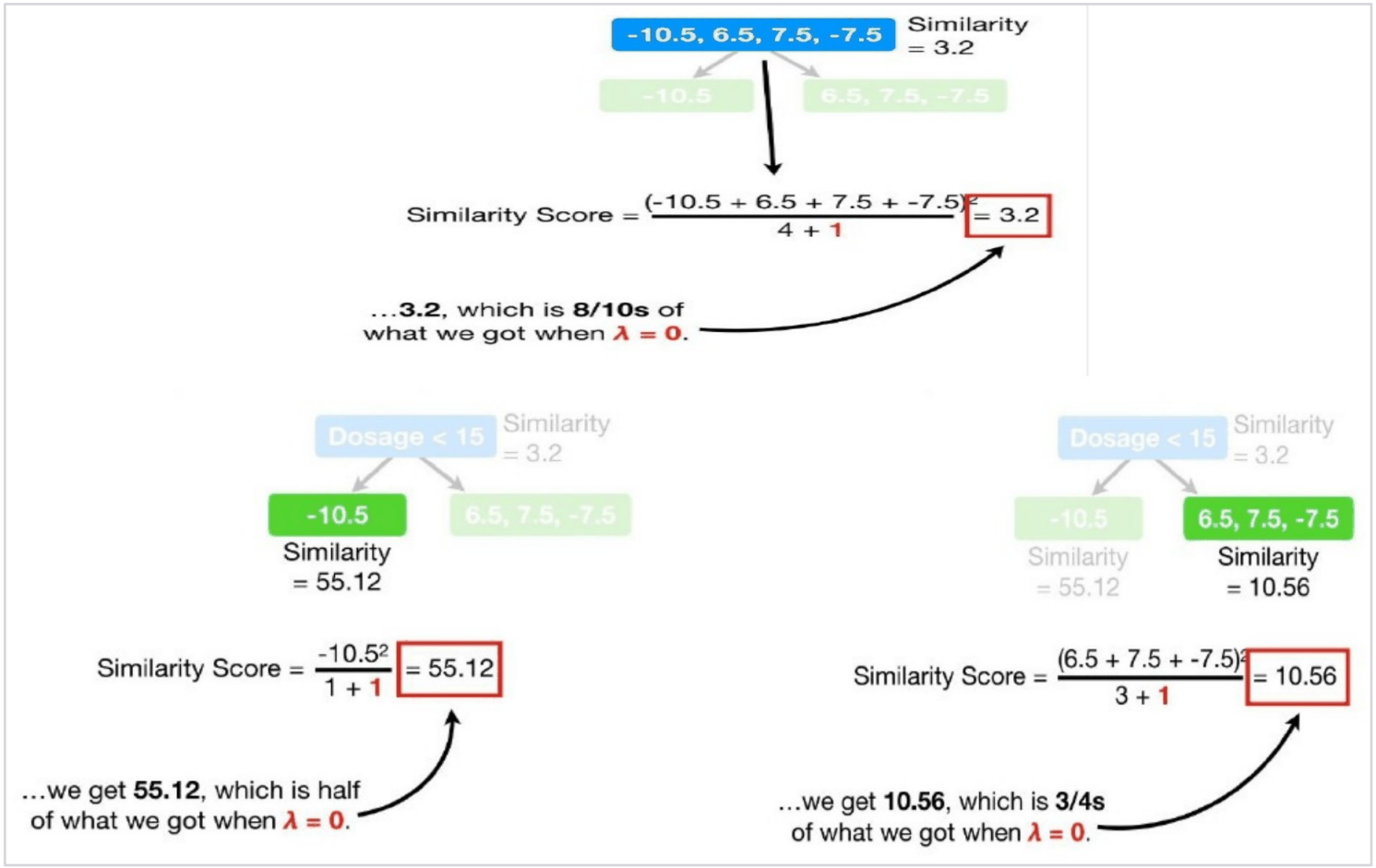
아래 가지를 남기면 뿌리의 이득(120.33)이 γ보다 작아도 뿌리는 남는다.

λ를 키우면 유사도가 줄어 가지치기가 쉬워진다

λ = 1로 다시 계산한 유사도와 이득

λ = 1의 효과

- 뿌리 유사도가 4에서 3.2로 줄어든다.
- 관측치가 적은 리프일수록 크게 준다. 110.25가 55.12로 절반이 된다.
- λ는 개별 관측치에 대한 예측 민감도를 낮추는 규제화다.

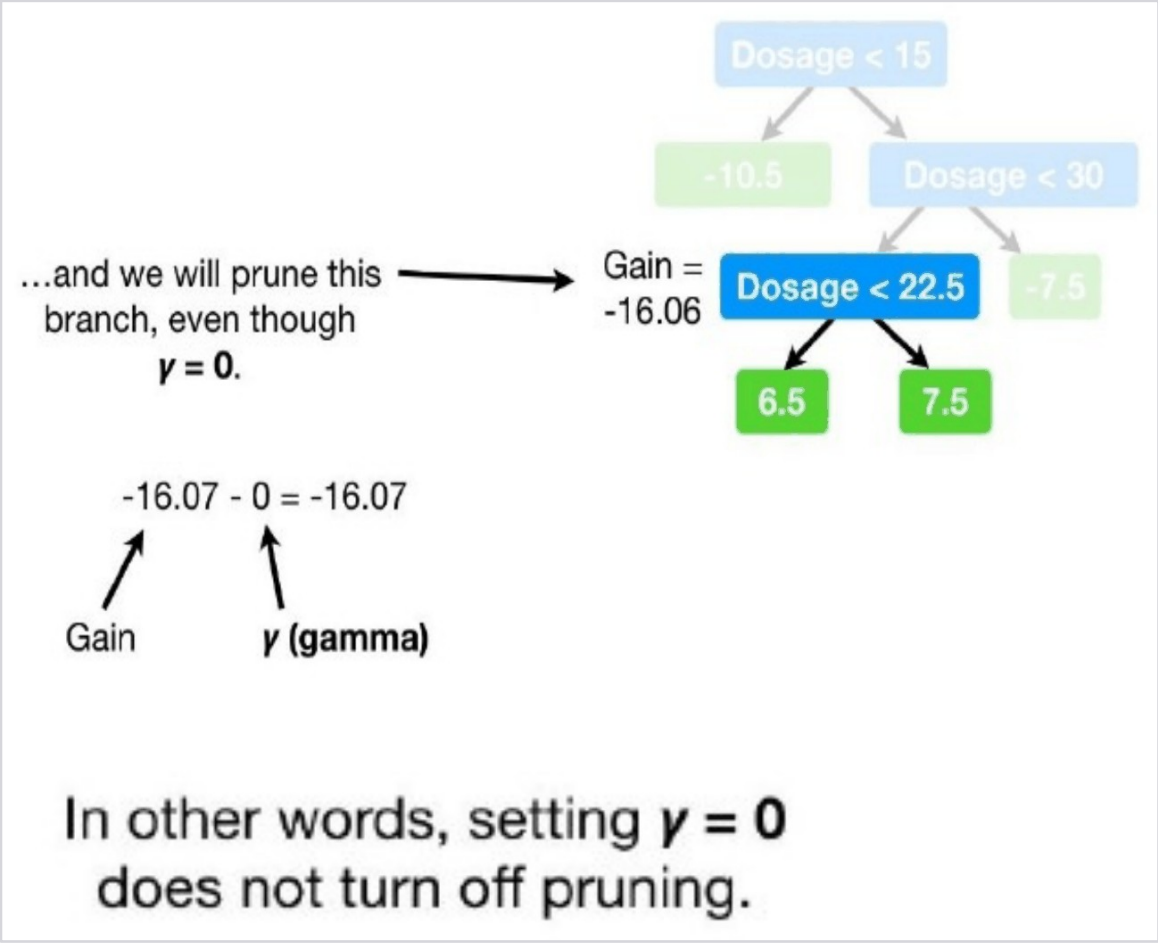


γ를 0으로 두어도 가지치기는 일어난다

λ = 1이면 이득 자체가 음수가 될 수 있다

읽는 법

- λ = 1에서 용량 22.5 미만 분할의 이득은 −16.06이다.
- γ = 0이어도 '이득 − γ'가 음수라 이 가지를 잘라 낸다.
- γ = 0은 가지치기를 끄는 설정이 아니다.



리프 출력값은 잔차 평균을 λ로 줄인 값이다

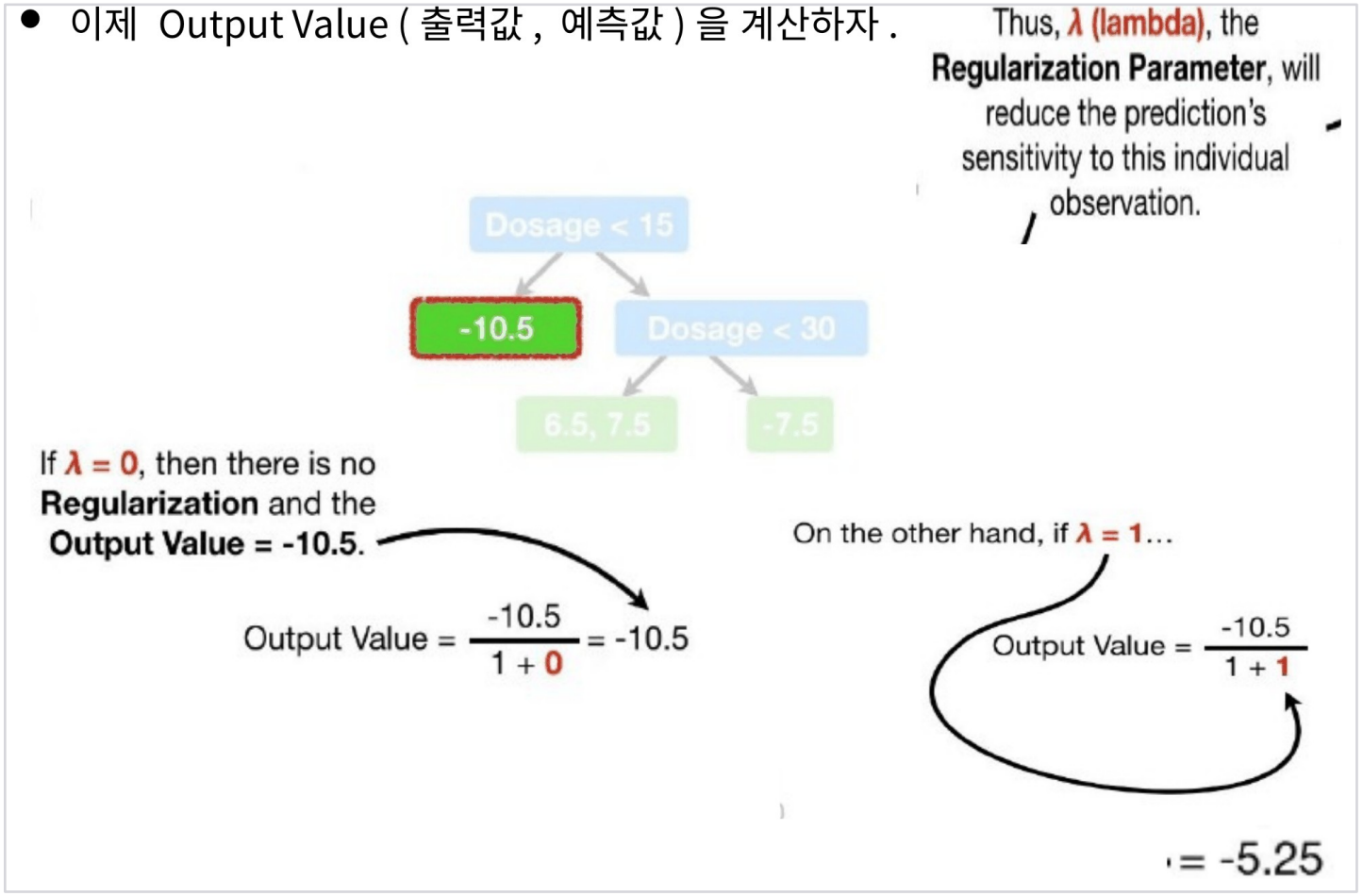
출력값 공식은 유사도 공식에서 분자의 제곱만 뺀 꼴이다

$$\text{출력값} = \frac{\text{잔차의 합}}{\text{잔차 개수} + \lambda}$$

λ에 따른 변화

- λ = 0이면 왼쪽 리프 출력은 −10.5다.
- λ = 1이면 −5.25로 줄어든다.
- 한 관측치에 휘둘리지 않게 출력을 줄인다.

- 이제 Output Value (출력값 , 예측값) 을 계산하자 .

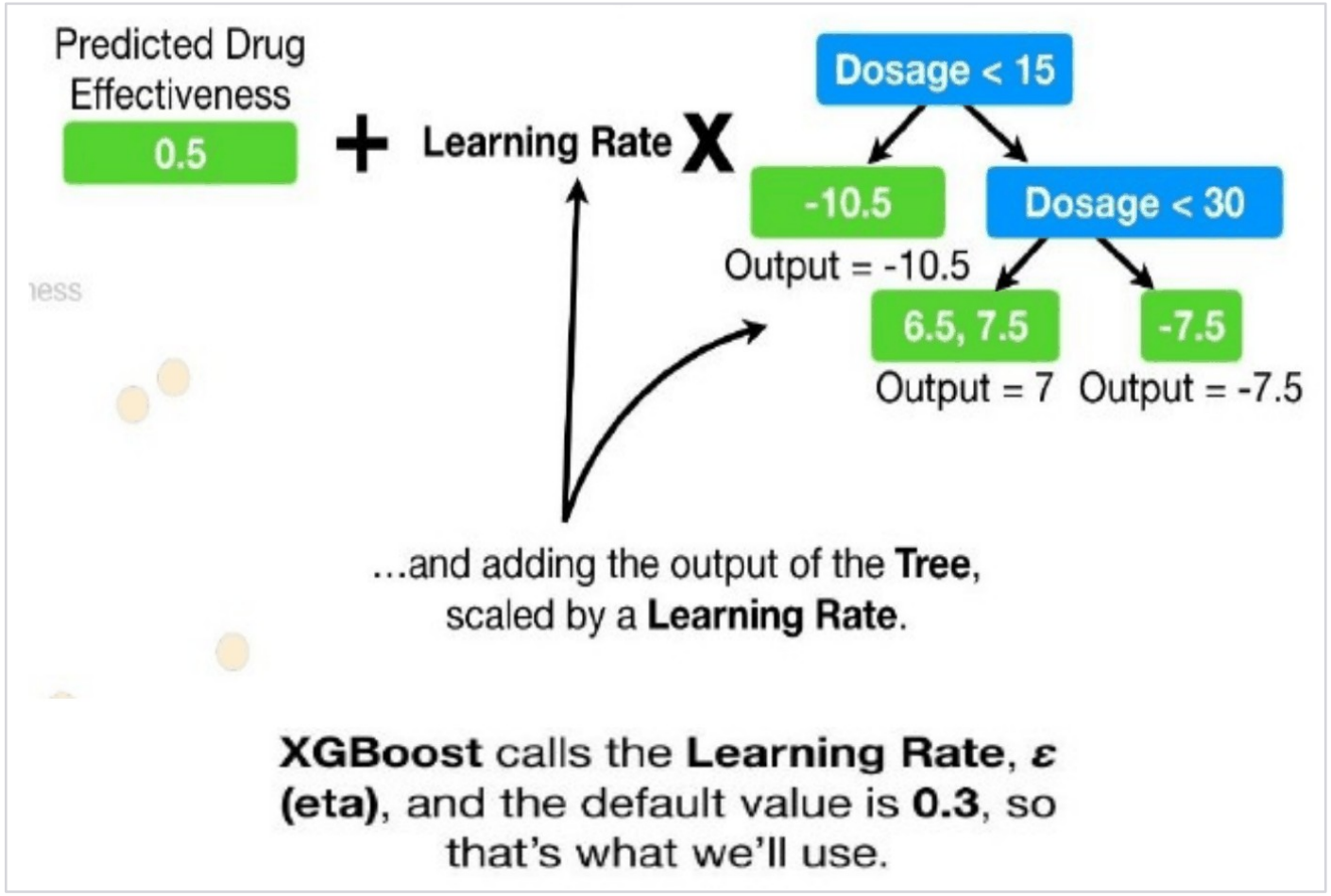


학습률 0.3만큼만 출력값을 반영한다

XGBoost의 학습률 eta 기본값은 0.3이다

읽는 법

- 첫 관측치의 새 예측은 0.5 + 0.3 × (−10.5) = −2.65다.
- 잔차가 −10.5에서 −7.35로 줄어 실제값에 조금 다가간다.
- 작은 걸음을 여러 번 걷는 것이 과적합을 막는다.

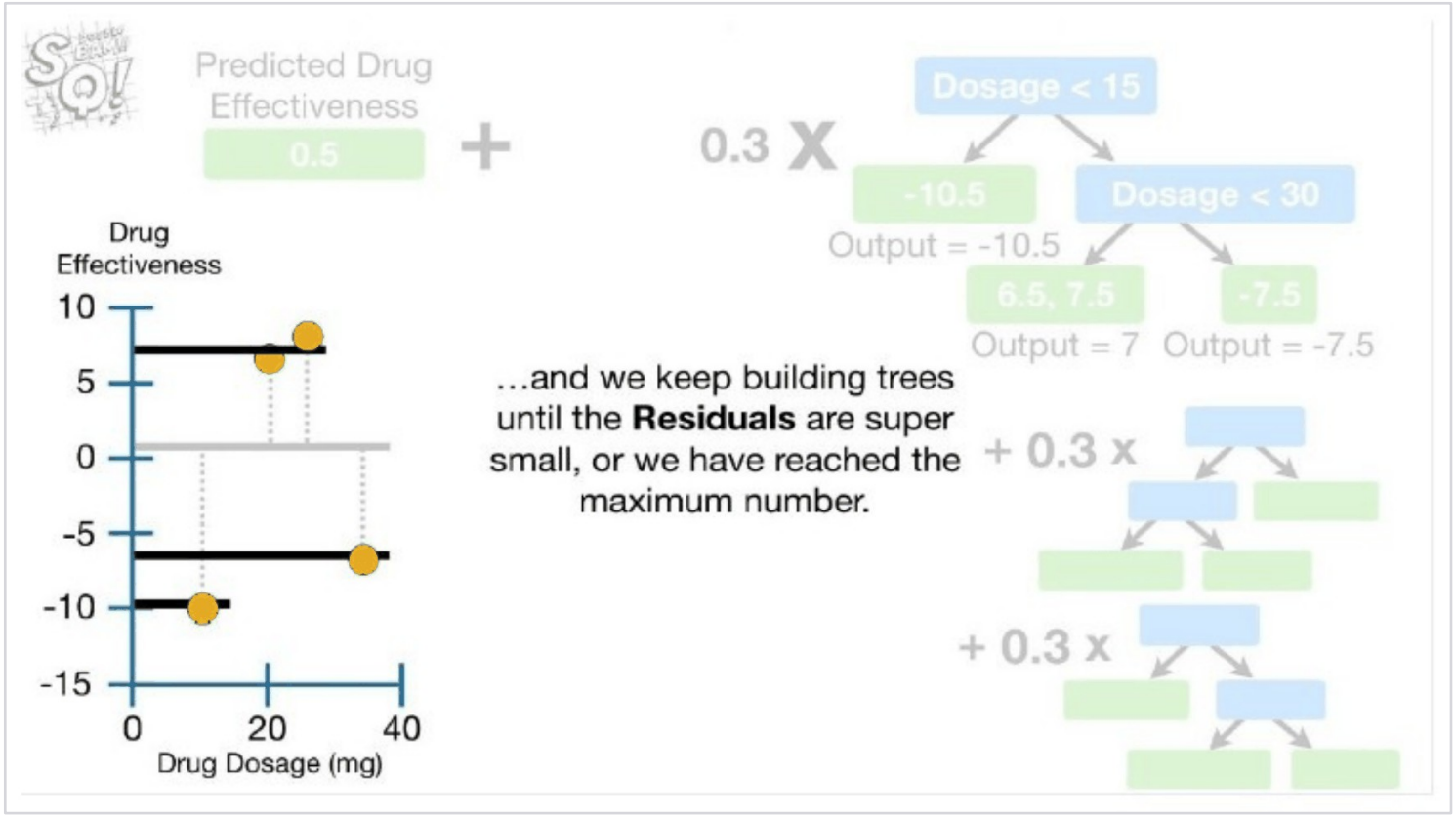


잔차가 충분히 작아질 때까지 트리를 더한다

새 잔차로 다음 트리를 만들고 학습률을 곱해 더한다

읽는 법

- 두 번째 트리는 줄어든 잔차를 학습한다.
- 잔차가 매우 작아지거나 트리 수가 최대에 이르면 멈춘다.
- 실무에서는 검증 오차가 더 줄지 않을 때 멈춘다(조기 종료).



회귀 트리 하나는 공식 네 개로 만든다

따라가기 정리 — 유사도로 묶고, 이득으로 고르고, γ로 자르고, 출력값을 정한다

① 유사도 — 잔차가 얼마나 비슷한가

$$\text{유사도} = \frac{(\text{잔차의 합})^2}{\text{잔차 개수} + \lambda}$$

② 이득 — 분할로 유사도가 얼마나 늘었나

$$\text{이득} = \text{유사도}_{\text{왼쪽}} + \text{유사도}_{\text{오른쪽}} - \text{유사도}_{\text{뿌리}}$$

③ 가지치기 — 이득이 γ보다 작으면 자른다

$$\text{이득} - \gamma < 0 \;\Rightarrow\; \text{가지 제거}$$

④ 출력값 — 리프가 내놓는 예측 조정분

$$\text{출력값} = \frac{\text{잔차의 합}}{\text{잔차 개수} + \lambda}$$

XGBoost는 대용량을 위해 여섯 가지 장치를 더했다

회귀 트리 계산 위에 얹은 근사 알고리즘과 시스템 최적화

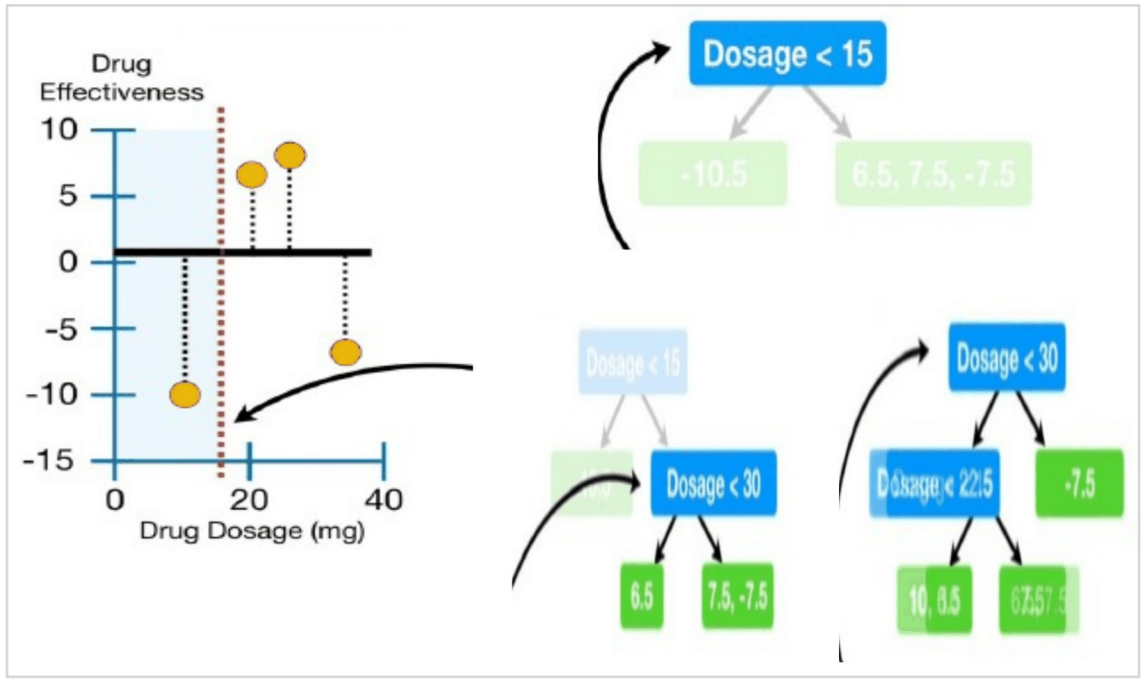
장치	하는 일
근사적 탐욕 알고리즘	모든 값 대신 분위수 경계만 분할 후보로 본다
병렬 학습	데이터를 나눠 여러 컴퓨터가 동시에 계산한다
가중 분위 스케치	헤시안을 가중치로 분위수를 나눠 근사 히스토그램을 만든다
희소 인지 분할	결측값이 갈 기본 방향을 학습한다
캐시 인지 접근	그래디언트를 CPU 캐시에 두어 계산을 빠르게 한다
외부 메모리 블록	디스크의 압축 블록을 나눠 읽어 메모리보다 큰 데이터를 다룬다

원본의 'More than 통계학' — 좋은 알고리즘에 공학적 장치가 더해져야 현업에서 쓰인다.

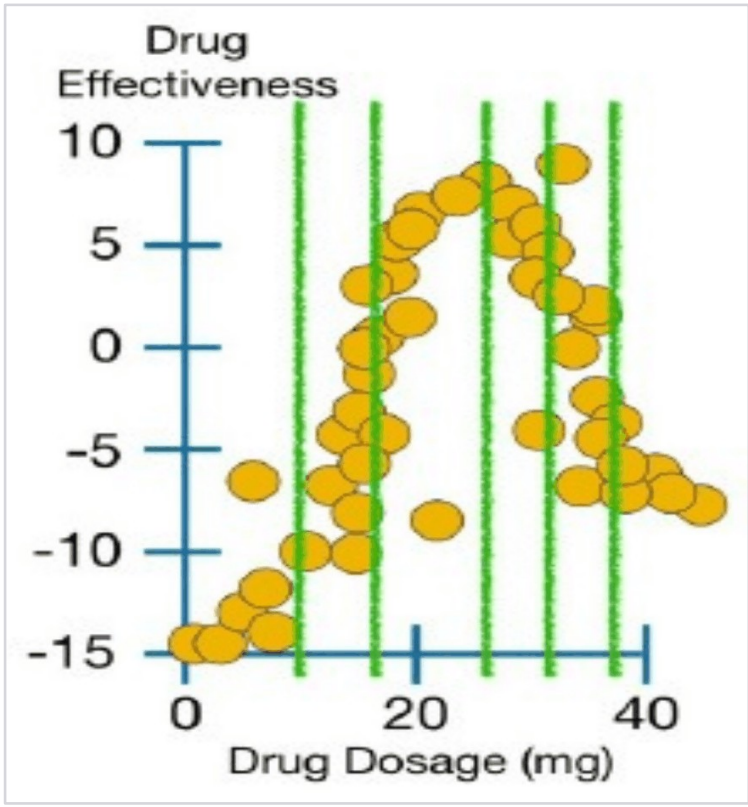
근사 탐욕 알고리즘은 분위수 경계만 시험한다

모든 경계를 다 보는 탐욕 알고리즘은 데이터가 크면 느리다

탐욕 알고리즘 — 한 단계씩 모든 경계를 시험한다



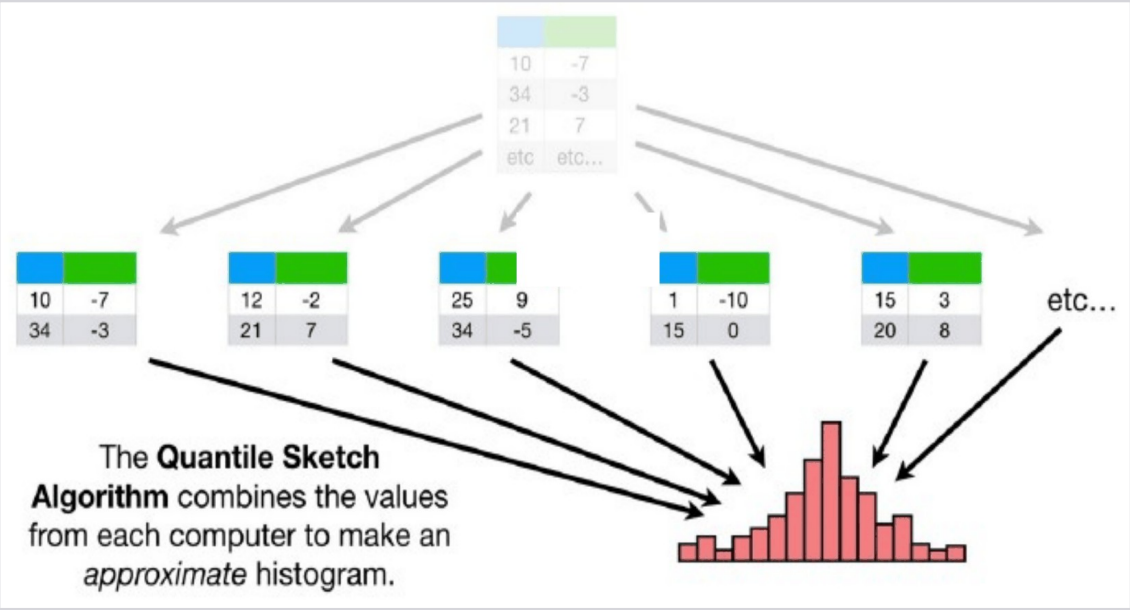
근사 탐욕 — 분위수 경계(초록 선)만 시험한다



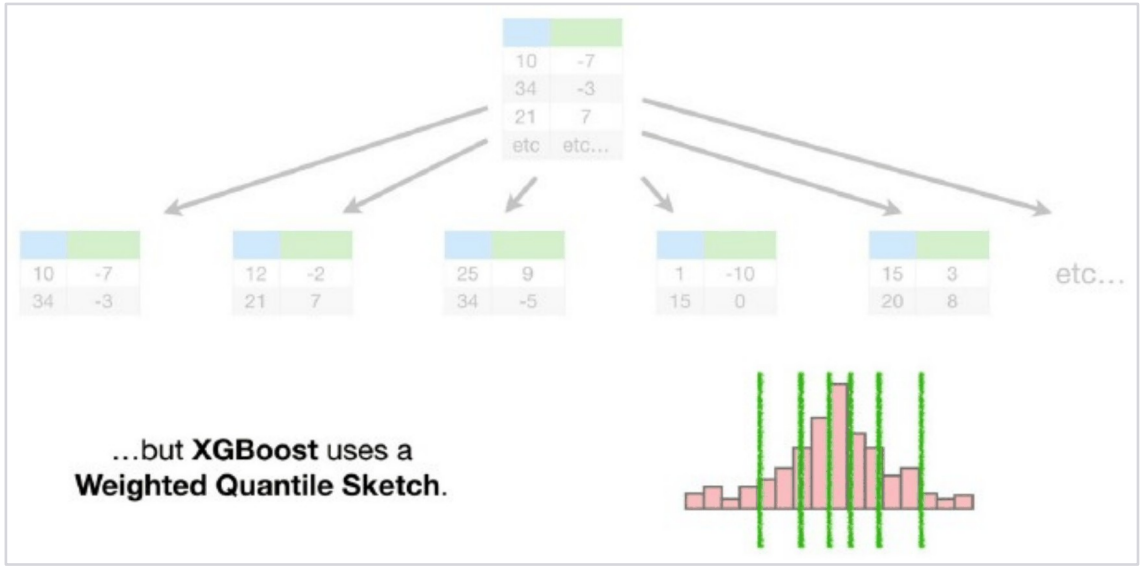
분위수는 데이터를 나눠 계산한 뒤 합친다

병렬 학습으로 나눈 조각마다 값을 요약하고 근사 히스토그램으로 합친다

분위 스케치 — 각 컴퓨터의 요약을 합쳐 근사 히스토그램을 만든다



XGBoost는 가중치를 반영한 가중 분위 스케치를 쓴다

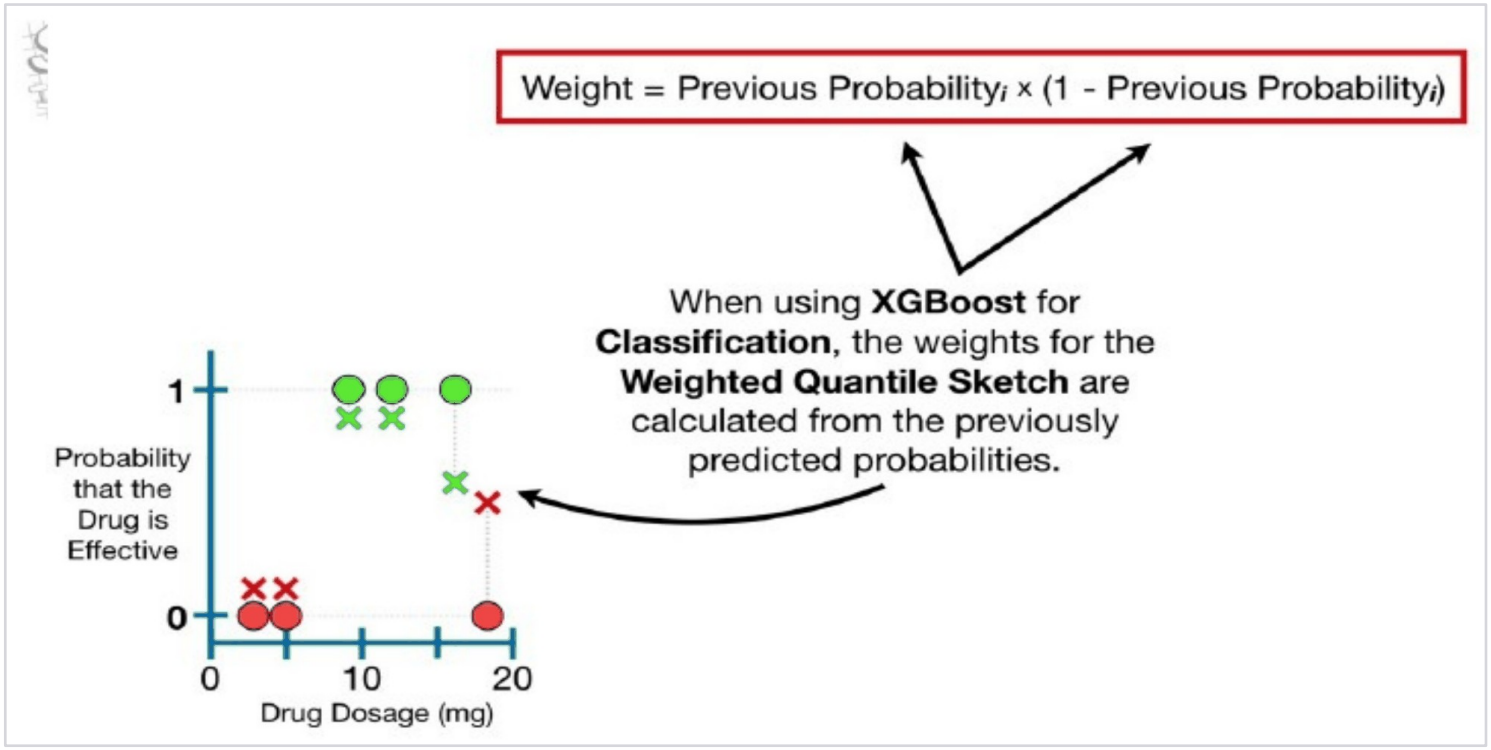


가중치는 이전 예측의 불확실성이다

분류에서 가중치는 이전 예측확률 × (1 − 이전 예측확률)이다

읽는 법

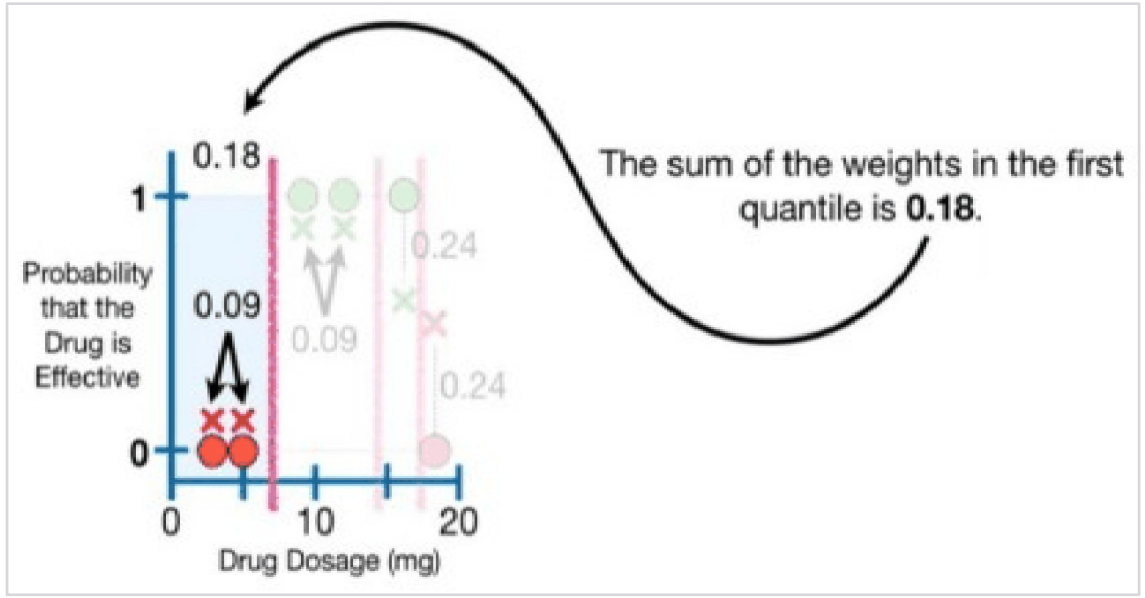
- 이 가중치는 로그 손실의 헤시안과 같다.
- 예측확률이 0.5에 가까울수록 가중치가 크다.
- 이미 확실하게 맞힌 관측치는 가중치가 작다.



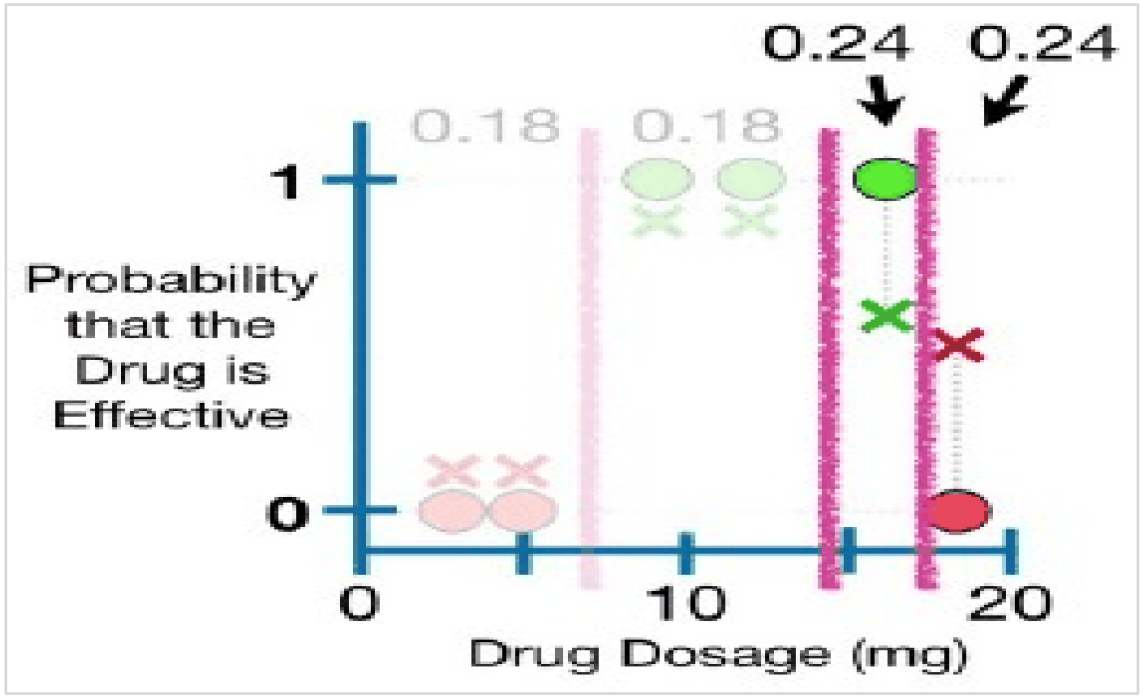
분위마다 가중치 합이 같도록 경계를 정한다

개수가 아니라 가중치로 나누면 불확실한 구간이 더 잘게 쪼개진다

첫 분위는 관측치 여럿의 가중치 합이 0.18이다



마지막 분위는 관측치 하나만으로 가중치가 0.24다

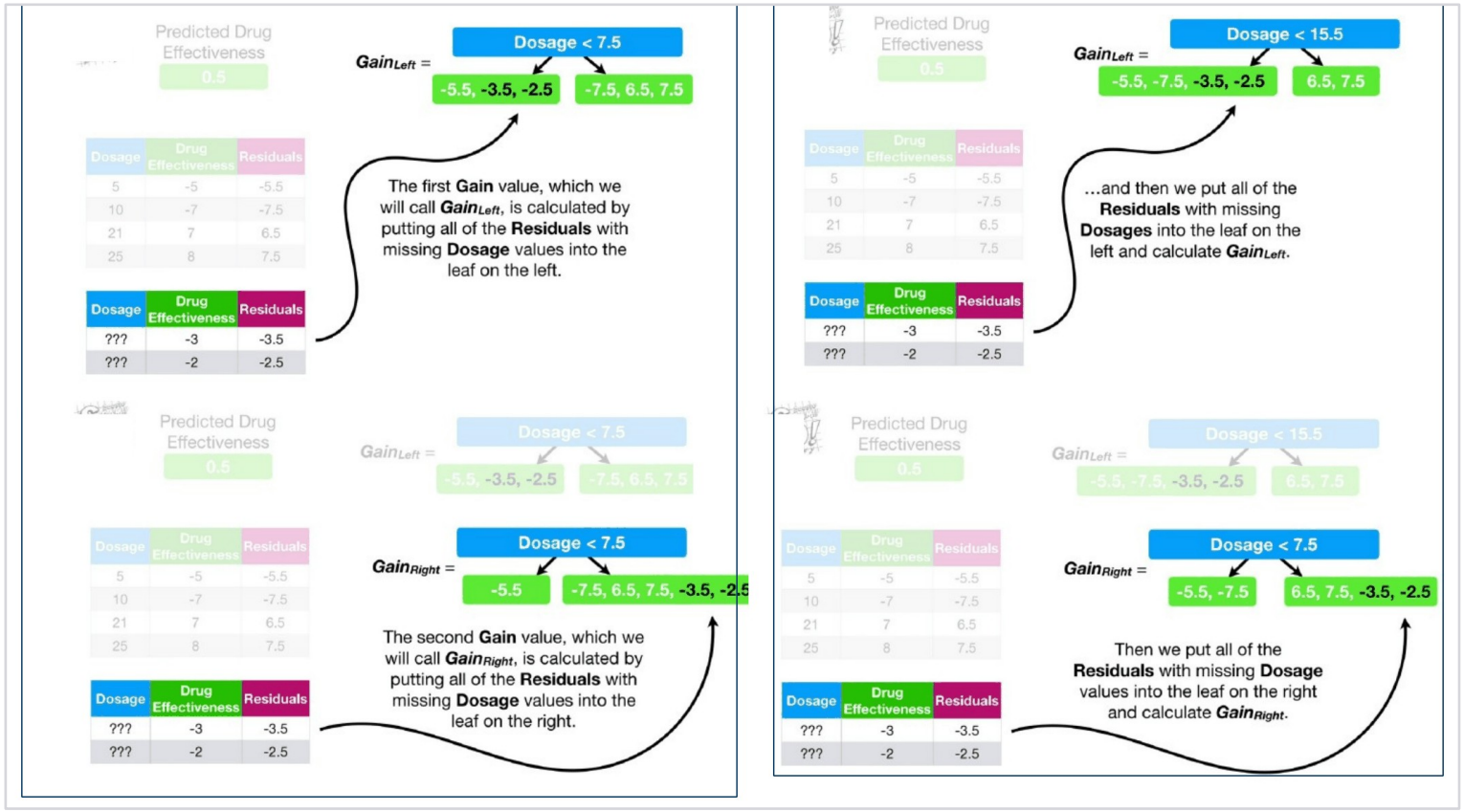


결측값은 이득이 더 큰 쪽으로 보낸다

희소 인지 분할(Sparsity-aware split) — 결측 관측치를 양쪽에 넣어 본다

읽는 법

- 결측 잔차를 왼쪽에 넣은 이득과 오른쪽에 넣은 이득을 비교한다.
- 더 큰 쪽이 그 분할의 기본 방향이 된다.
- 원-핫 인코딩으로 생긴 0이 많은 데이터에도 같은 원리를 쓴다.

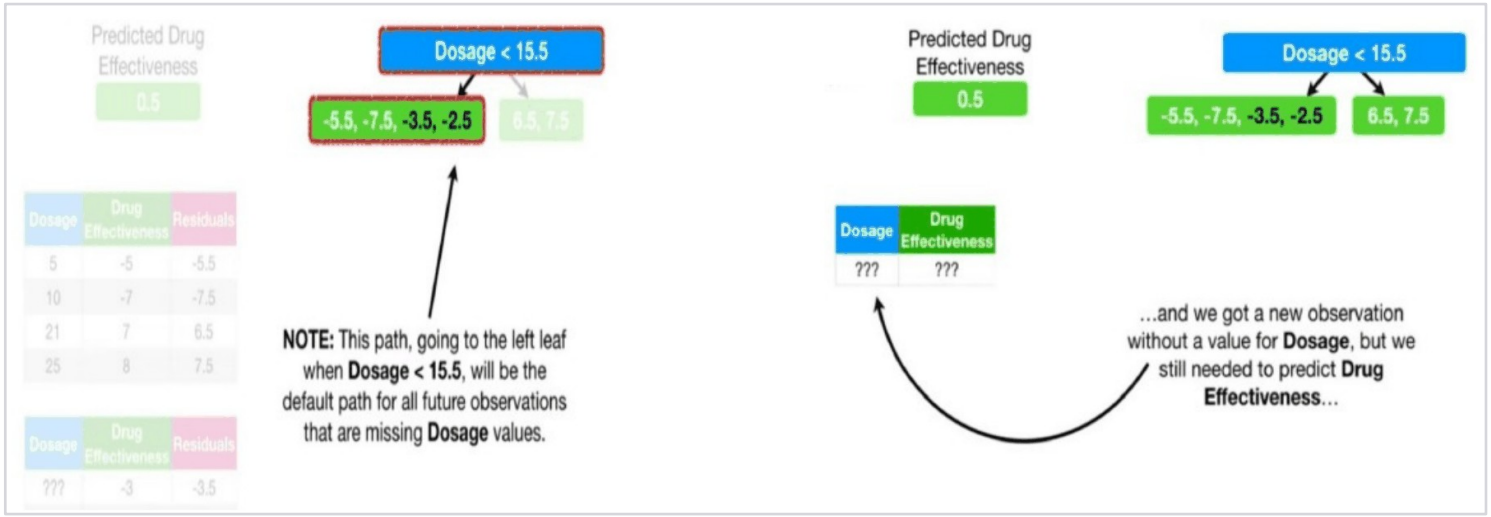


새 결측 관측치도 학습된 기본 방향을 따른다

학습 때 정한 방향으로 예측 시점의 결측값을 처리한다

읽는 법

- 학습 데이터의 결측값은 기본 방향 쪽 리프로 간다.
- 새 관측치의 용량이 비어 있어도 같은 경로로 예측한다.
- 결측값 대치(imputation) 없이 바로 학습할 수 있다.

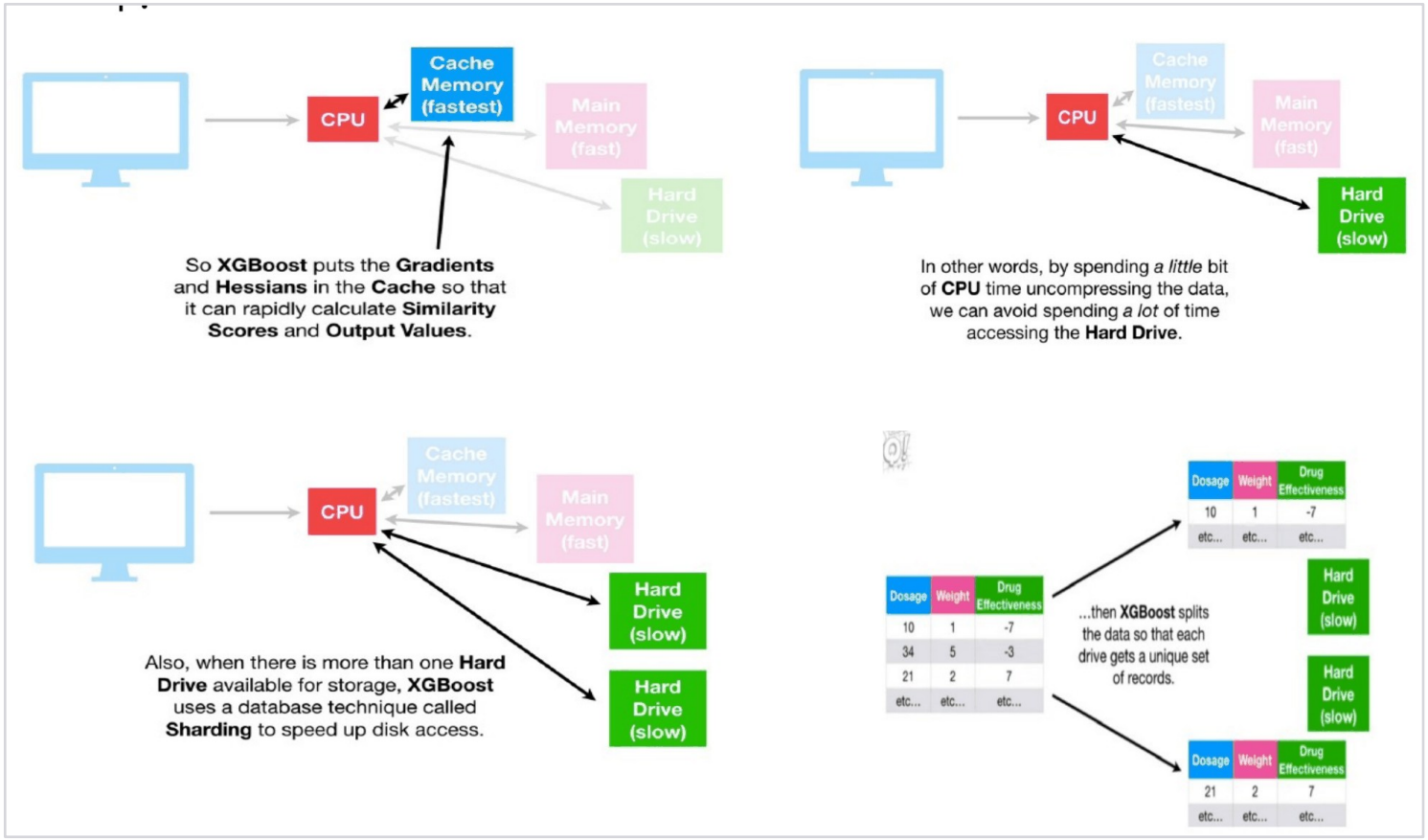


캐시와 디스크를 고려해 계산 속도를 높인다

캐시 인지 접근과 외부 메모리 계산(out-of-core)

읽는 법

- 그래디언트와 헤시안을 CPU 캐시에 두고 유사도를 빠르게 계산한다.
- 디스크를 쓸 때는 블록을 압축해 읽기량을 줄이고, 디스크가 여럿이면 나눠 저장한다.
- 3.0부터 외부 메모리 학습이 GPU에서도 빨라졌다.



행과 열을 무작위로 골라 트리마다 다르게 학습한다

subsample과 colsample_bytree — 랜덤 포레스트에서 빌린 아이디어

읽는 법

- 트리마다 관측치 일부만 쓴다.
- 트리마다 변수 일부만 쓴다.
- 속도가 빨라지고 트리 사이 상관이 줄어 과적합이 줄어든다.



02

PART 2 · LightGBM

LightGBM — 덜 보고 빨리 찾는 부스팅

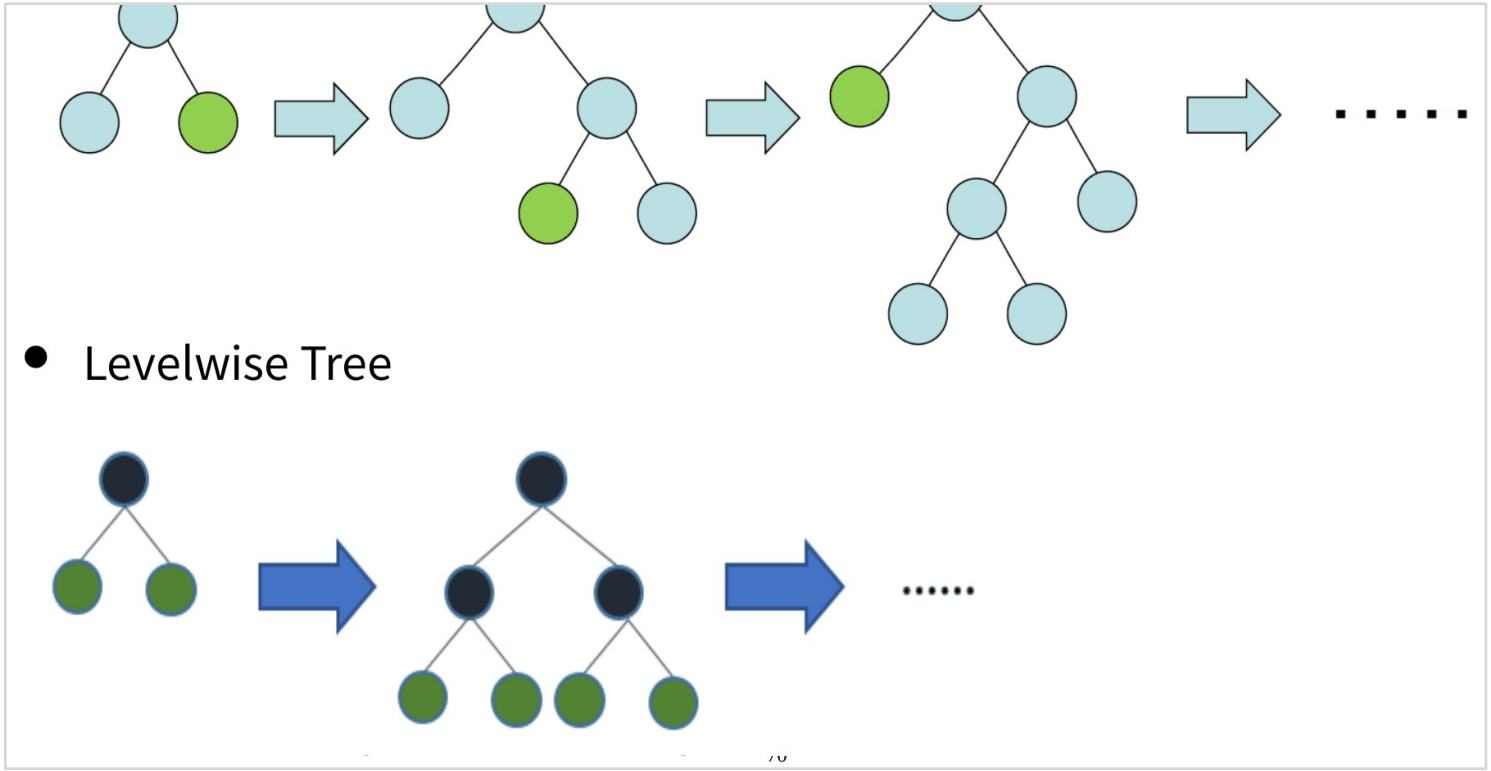
리프 중심 성장, GOSS, EFB로 속도와 메모리를 줄이는 방법

LightGBM 트리는 손실이 큰 리프부터 깊게 판다

Leaf-wise 성장 — 위는 LightGBM, 아래는 XGBoost의 level-wise

읽는 법

- 이득이 가장 큰 리프 하나만 골라 나눈다.
- 덜 중요한 쪽은 자라지 않아 같은 리프 수에서 손실이 더 빨리 준다.
- 대신 트리가 깊어지기 쉬워 num_leaves와 max_depth로 막는다.

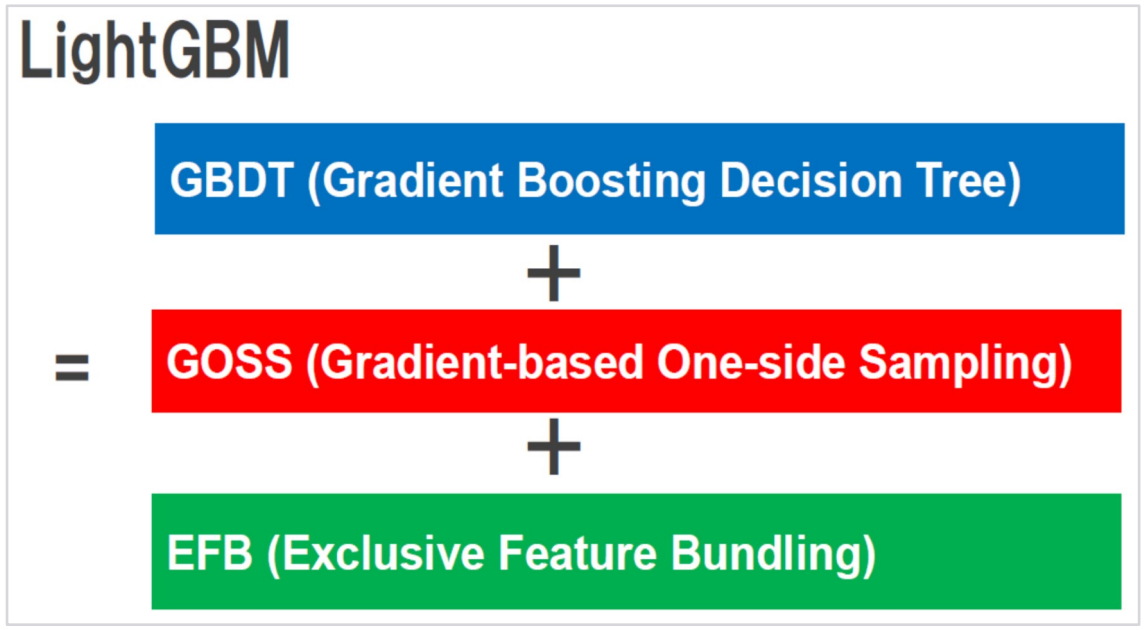


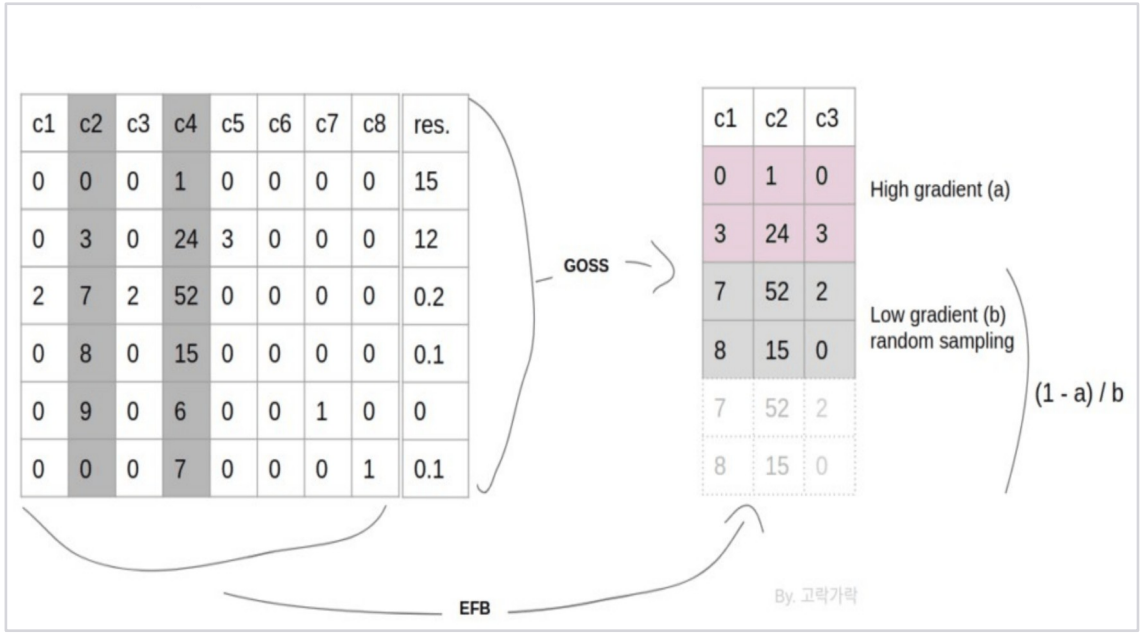
LightGBM은 GBDT에 두 가지 기법을 더했다

GOSS는 계산할 행을 줄이고 EFB는 계산할 열을 줄인다

LightGBM = GBDT + GOSS + EFB

그래디언트로 행을 고르고, 배타적인 열을 묶는다



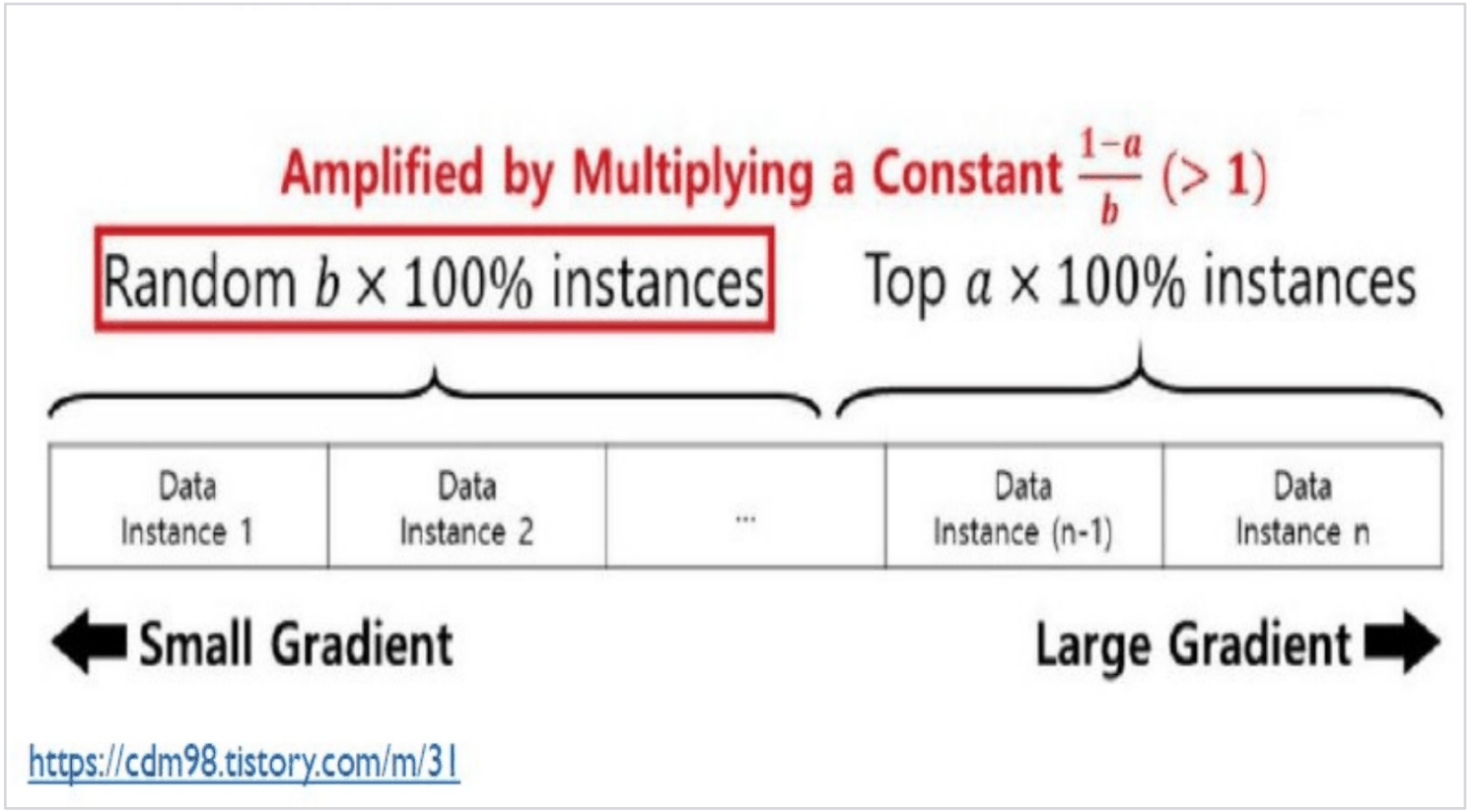


GOSS는 그래디언트 크기로 관측치를 고른다

Gradient-based One-Side Sampling — 그래디언트가 큰 관측치는 모두 쓰고, 나머지는 일부만 뽑는다

읽는 법

- 그래디언트가 큰 관측치는 아직 덜 학습된 관측치다.
- 상위 a 비율은 모두 쓰고, 나머지에서 전체의 b 비율만큼 무작위로 뽑는다.
- 뽑은 관측치에 상수를 곱해 원래 분포를 보정한다.



a = 0.2, b = 0.1이면 데이터 30%로 분할을 찾는다

GOSS 샘플링 예시 — 작은 그래디언트 쪽은 표본에 가중치를 줘 보정한다 (※ 예시 데이터)



$$\frac{1-a}{b}$$

보정 상수

- 예시에서는 0.8을 0.1로 나눈 8이다.
- 작은 그래디언트 쪽 합이 원래 크기로 복원된다.

EFB는 동시에 0이 아닌 일이 드문 변수를 묶는다

Exclusive Feature Bundling — 두 변수가 동시에 0이 아닌 횟수(충돌)를 센다

읽는 법

- 왼쪽 표는 관측치 10개, 변수 5개의 희소 데이터다.
- 오른쪽 표는 변수 쌍마다 충돌 횟수를 센 것이다.
- 충돌이 적은 변수끼리는 하나로 합쳐도 정보 손실이 작다.

	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

	x_1	x_2	x_3	x_4	x_5
x_1	-	6	2	1	6
x_2	6	-	1	1	6
x_3	2	1	-	3	4
x_4	1	1	3	-	3
x_5	6	6	4	3	-

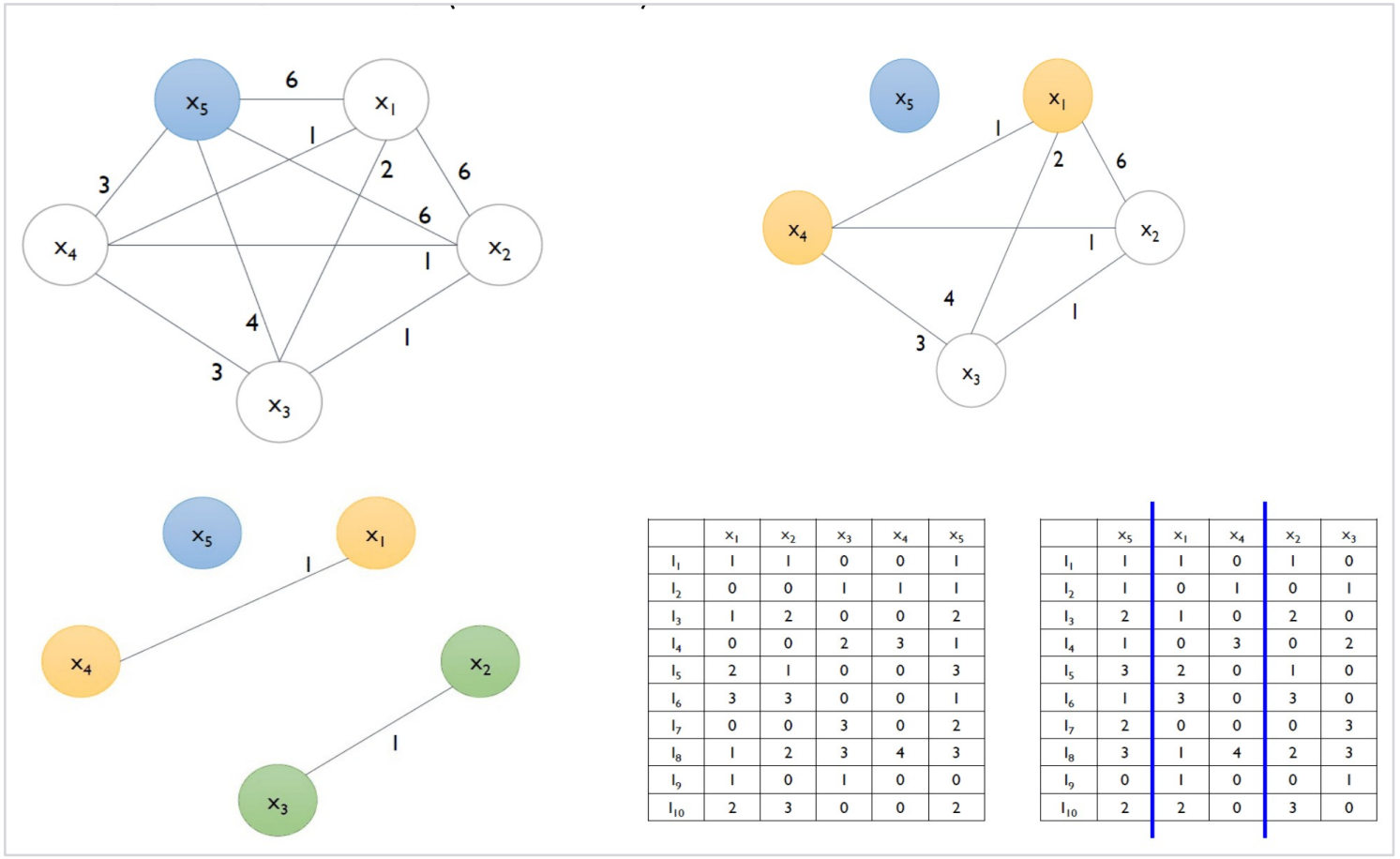
	x_5	x_1	x_2	x_3	x_4
d	19	15	14	10	8

충돌 그래프를 색칠하듯 변수 묶음을 정한다

충돌 허용 기준 0.2 — 연결이 많은 변수부터 묶음에 배정한다

읽는 법

- 변수는 꼭짓점, 충돌 횟수는 간선 가중치다.
- 기준을 넘는 간선으로 이어진 변수는 서로 다른 묶음에 넣는다.
- 변수 5개가 더 적은 묶음으로 줄어든다.



	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

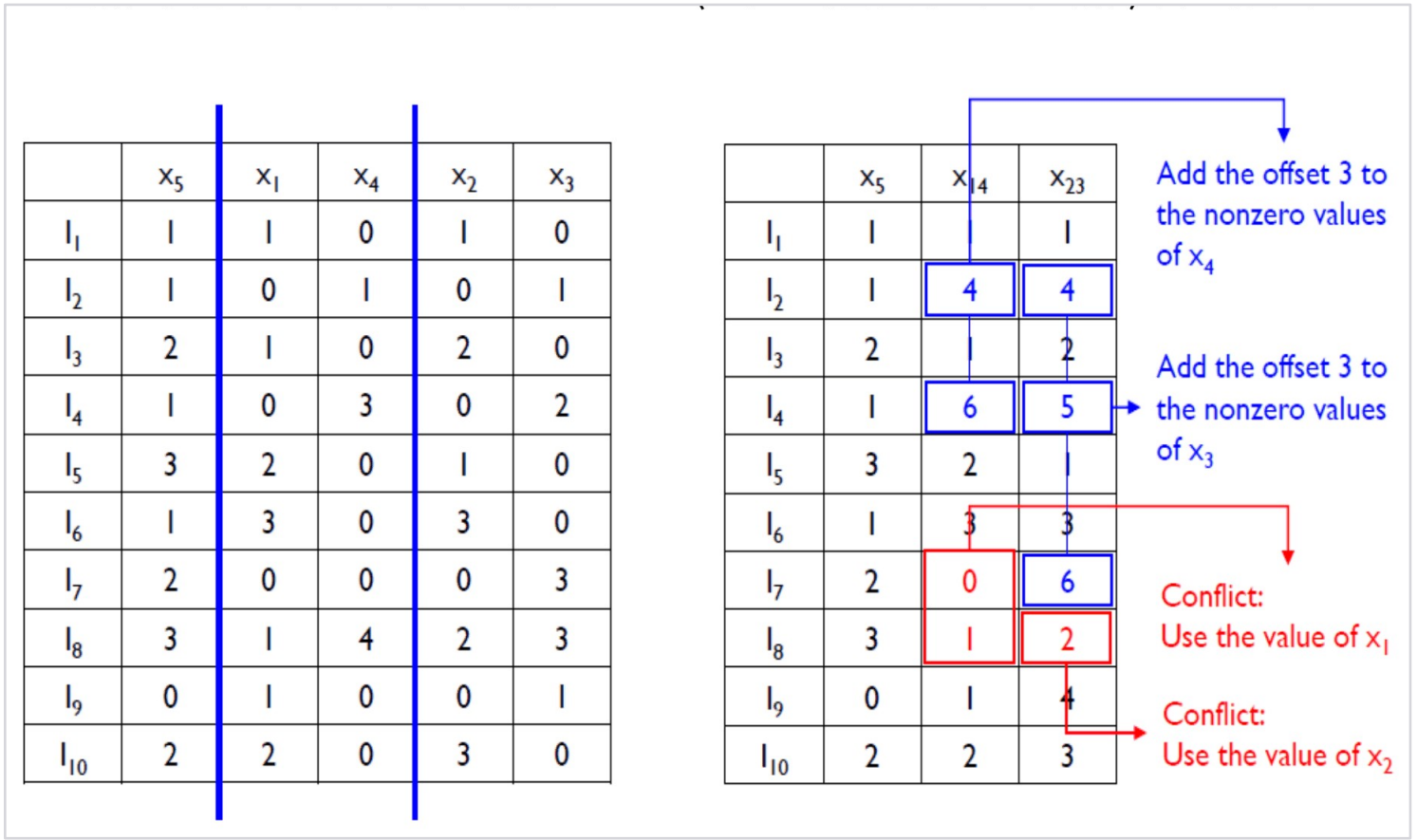
	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0

묶을 때는 오프셋을 더해 원래 변수를 구별한다

기준 변수의 최댓값만큼 더해 값의 범위가 겹치지 않게 한다

읽는 법

- x4의 0이 아닌 값에 3을 더해 x1과 범위를 나눈다.
- 두 변수가 동시에 0이 아니면 충돌이며, 한쪽 값을 택한다.
- 묶은 변수 하나로 히스토그램을 만들어 계산량을 줄인다.



	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0

	x_5	x_{14}	x_{23}
I_1	1	1	1
I_2	1	4	4
I_3	2	1	2
I_4	1	6	5
I_5	3	2	1
I_6	1	3	3
I_7	2	0	6
I_8	3	1	2
I_9	0	1	4
I_{10}	2	2	3

LightGBM 튜닝은 num_leaves에서 시작한다

리프 중심 성장의 과적합을 막는 핵심 파라미터 (LightGBM 4.7 문서 기준)

파라미터	기본값	조절 요령
num_leaves	31	가장 중요하다. max_depth로 만들 수 있는 리프 수보다 작게 둔다
max_depth	−1 (제한 없음)	트리가 지나치게 깊어질 때 함께 제한한다
min_data_in_leaf	20	키우면 작은 리프가 줄어 과적합이 준다
max_bin	255	줄이면 빨라지지만 분할 후보가 거칠어진다
feature_fraction	1.0	0.7~0.9로 낮춰 트리마다 변수를 다르게 쓴다

2026년 7월 4.7.0부터 polars 입력과 NVIDIA 다중 GPU 학습을 지원한다.

03

PART 3 · CatBoost

CatBoost — 순서로 누수를 막는 부스팅

타깃 인코딩의 누수, Ordered Boosting, 대칭 트리

잔차를 같은 데이터로 학습하면 예측이 치우친다

기존 부스팅의 절차와 CatBoost가 지적한 문제

기존 부스팅의 절차

- 실제값 평균과 실제값의 차이로 잔차를 구한다.
- 잔차를 학습하는 모델을 만든다.
- 예측에 학습률을 곱해 더하고, 이를 반복한다.

문제점

- 학습 속도가 느리다.
- 같은 관측치로 잔차를 계산하고 다시 학습해 과적합된다.
- 논문은 이를 예측 이동(prediction shift)이라 부른다.

CatBoost는 인코딩과 잔차 계산 모두에 '앞선 행만 쓴다'는 순서 원칙을 적용한다.

타깃 인코딩은 범주를 타깃 평균으로 바꾼다

Target(Mean) Encoding 예 — feature 1로 그날의 최고기온(class_label)을 예측한다

time	feature 1	class_label
sunday	sunny	35
monday	sunny	32
tues	cloudy	15
wed	cloudy	14
thurs	mostly_cloudy	10
fri	cloudy	20
sat	cloudy	25

계산과 문제

- cloudy를 그 범주의 class_label 평균 (15 + 14 + 20 + 25) / 4 = 18.5로 바꾼다.
- 범주가 많아도 열이 늘지 않는다.
- 그러나 예측할 타깃이 입력에 들어가 데이터 누수가 생긴다.

자기 타깃까지 쓰면 인코딩만으로 정답이 보인다

StatQuest 예 — 인코딩한 Color < 0.42 하나로 Troll 2 선호가 완벽히 갈린다

읽는 법

- 행 수가 적은 범주에서는 인코딩 값이 곧 자기 타깃이다.
- 트리는 이 값 하나로 학습 데이터를 완벽히 나눈다.
- 새 데이터에서는 통하지 않는 전형적인 과적합이다.
- 한 행을 빼고 평균을 내는 Leave-One-Out 방식도 문제가 남는다.



	Favorite Color	Height (m)	Loves Troll 2
Subset A	0.33	1.77	1
Subset B	0.5	1.32	0
Subset C	0.33	1.81	1
Subset D	0.5	1.56	0
Subset E	0.33	1.64	1
Subset F	0.5	1.61	0
Subset G	0.5	1.73	0

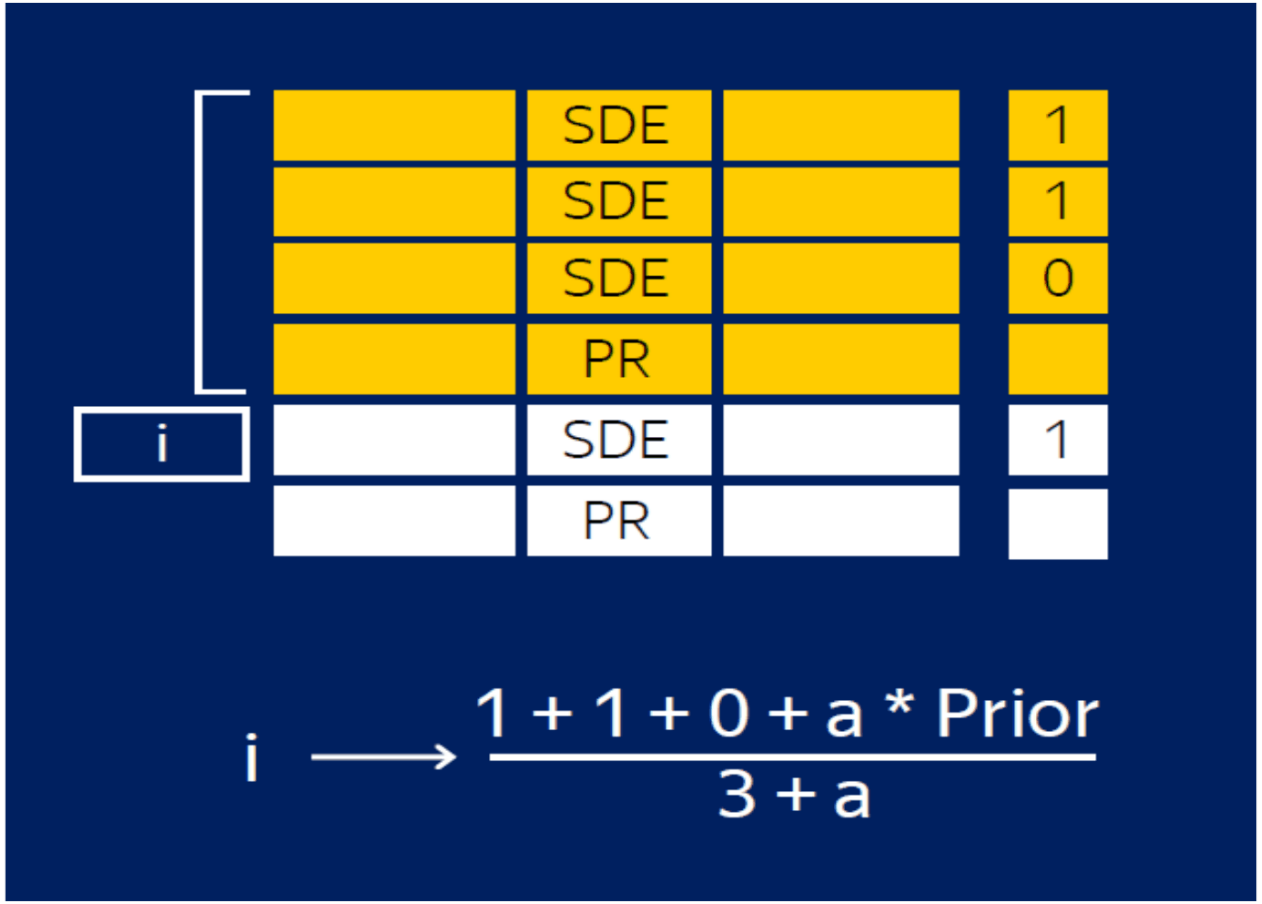
Favorite Color	Height (m)	Loves Troll 2
0	1.77	1
0	1.32	0
0	1.81	1
0	1.56	0
0	1.64	1
0	1.61	0
0	1.73	0

Ordered TS는 앞선 행의 타깃만으로 인코딩한다

Target Based Statistics — 사전값 Prior와 가중치 a로 평균을 안정시킨다

읽는 법

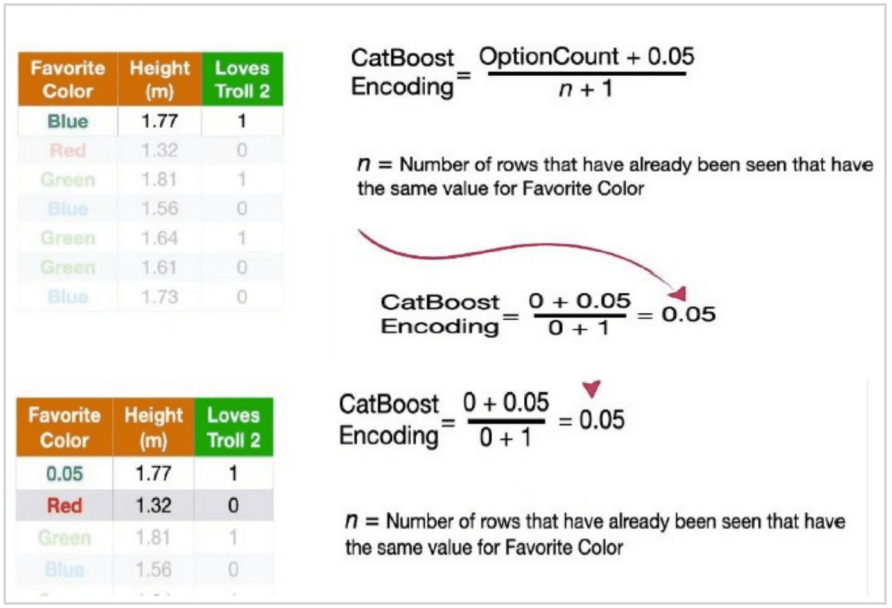
- i번째 행은 자기보다 앞에 있는 같은 범주 행만 본다.
- SDE의 앞선 타깃 1, 1, 0에 Prior를 섞어 평균을 낸다.
- 앞 예에서 금요일 cloudy는 (15 + 14) / 2 = 14.5, 토요일은 16.3이 된다.

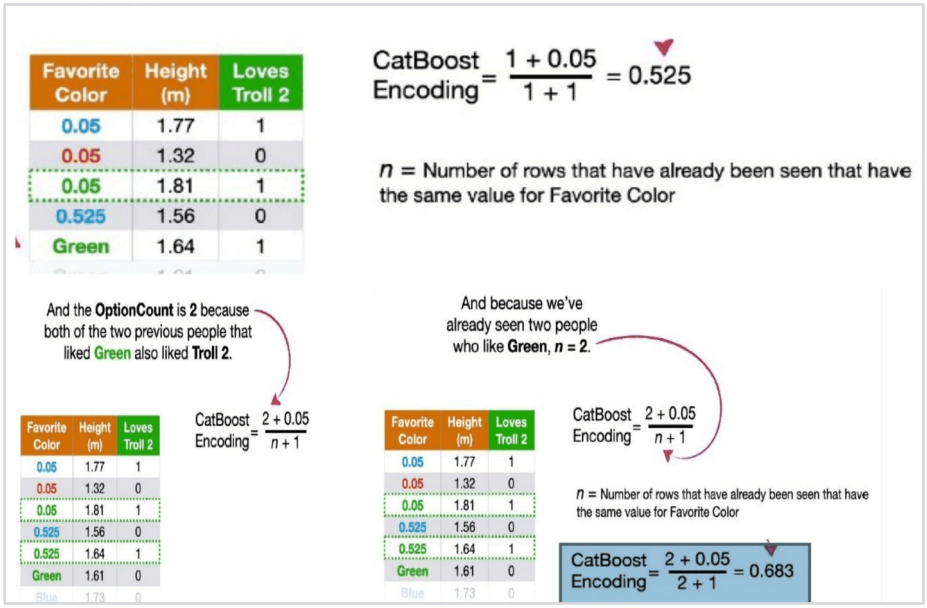


StatQuest 예제는 사전값으로 0.05를 쓴다

CatBoost 인코딩 공식 — 앞서 본 같은 범주 행만 센다

$$\text{CatBoost 인코딩} = \frac{\overbrace{\text{OptionCount}}^{\text{앞서 본 같은 범주의 양성 수}} + 0.05}{\underbrace{n}_{\text{앞서 본 같은 범주의 행 수}} + 1}$$





같은 범주라도 순서에 따라 값이 달라진다

그래서 Ordered Target Encoding이라 부른다

읽는 법

- 첫 Blue는 0.05, 다음 Blue는 앞선 Blue 한 명의 타깃을 반영해 0.525가 된다.
- 자기 타깃은 절대 쓰지 않아 누수가 없다.
- CatBoost는 행 순서를 무작위로 섞어 순서 효과를 평균낸다.

Favorite Color	Height (m)	Loves Troll 2
0.05	1.77	1
0.05	1.32	0
0.05	1.81	1
0.525	1.56	0
0.525	1.64	1
0.683	1.61	0
0.35	1.73	0

$$\text{CatBoost Encoding} = \frac{1 + 0.05}{2 + 1} = 0.35$$

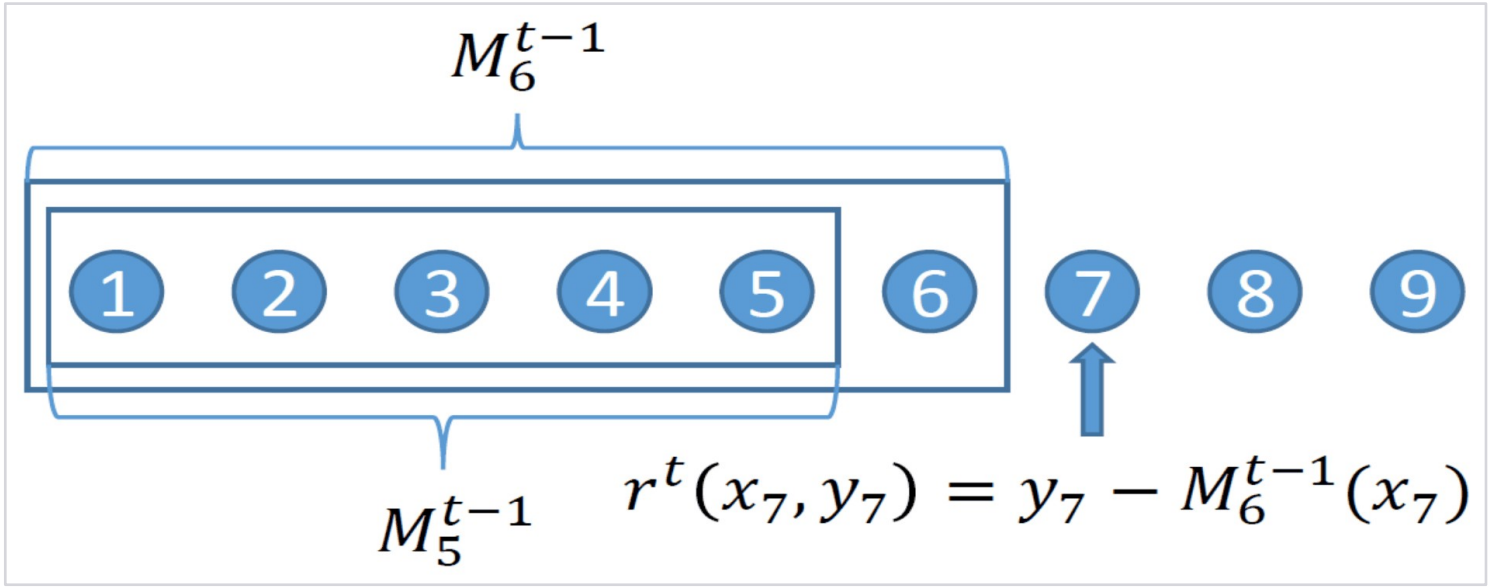
n = Number of rows that have already been seen that have the same value for Favorite Color

잔차도 앞선 관측치의 모델로만 계산한다

Ordered Boosting — 7번째 잔차는 6번째까지로 만든 모델이 계산한다

읽는 법

- x1의 잔차로 모델을 만들어 x2의 잔차를 예측한다.
- x1, x2로 모델을 만들어 x3, x4를 예측하는 식으로 늘려간다.
- 자기 자신을 본 모델로 잔차를 구하지 않아 예측 이동을 막는다.

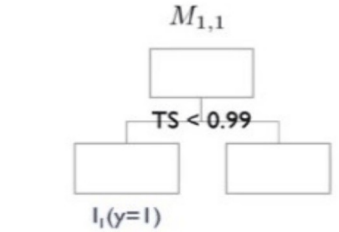


실제 구현은 그래디언트를 순서대로 누적한다

트리 구조는 하나이고, 리프 값은 행마다 앞선 행만으로 계산한다

첫 행은 그래디언트와 리프 값을 0으로 시작한다

	...	x^j	...	TS	y	G	Δ
I_1	...	A	...	0.000	I	0	0
I_2	...	B	...	1.000	I		
I_3	...	C	...	1.000	I		
I_4	...	A	...	1.000	0		
I_5	...	B	...	0.977	I		
I_6	...	C	...	0.982	I		
I_7	...	B	...	0.992	0		
I_8	...	C	...	0.986	I		
I_9	...	C	...	0.992	0		
I_{10}	...	C	...	0.748	I		



$G(I_1) = 0$

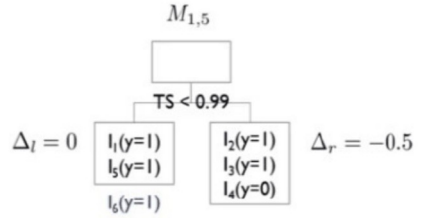
$\Delta(I_1) = 0$

Lecturer's guess

- Initialize both the first gradient and leaf value to 0

다섯째 행의 리프 값은 앞선 같은 리프 행들의 평균이다

	...	x^j	...	TS	y	G	Δ
I_1	...	A	...	0.000	I	0	0
I_2	...	B	...	1.000	I	0	0
I_3	...	C	...	1.000	I	-I	0
I_4	...	A	...	1.000	0	0	-0.5
I_5	...	B	...	0.977	I	-I	0
I_6	...	C	...	0.982	I	-I	-0.5
I_7	...	B	...	0.992	0		
I_8	...	C	...	0.986	I		
I_9	...	C	...	0.992	0		
I_{10}	...	C	...	0.748	I		



$$G(I_6) = grad_{1,\sigma_1(6)-1}(I_6)$$
$$= \Delta_l - y_6 = 0 - 1 = -1$$

$$\Delta(I_6) = \frac{1}{2}(grad_{1,\sigma_1(6)-1}(I_1) + grad_{1,\sigma_1(6)-1}(I_5))$$
$$= \frac{1}{2}(0 - 1) = -0.5$$

모든 행을 지나면 코사인 유사도로 분할을 평가한다

순서대로 계산한 리프 값과 실제 그래디언트가 같은 방향인지 본다

읽는 법

- 두 벡터의 코사인 유사도가 분할 후보의 점수다.
- 후보 분할마다 이 점수를 계산해 가장 좋은 분할을 고른다.
- 예제의 후보 TS < 0.99의 값은 0.3575다.

	...	x^j	...	TS	y	G	Δ
I_1	...	A	...	0.000	1	0	0
I_2	...	B	...	1.000	1	0	0
I_3	...	C	...	1.000	1	-1	0
I_4	...	A	...	1.000	0	0	-0.5
I_5	...	B	...	0.977	1	-1	0
I_6	...	C	...	0.982	1	-1	-0.5
I_7	...	B	...	0.992	0	-0.5	-0.33
I_8	...	C	...	0.986	1	-1.5	-0.67
I_9	...	C	...	0.992	0	-0.33	-0.38
I_{10}	...	C	...	0.748	1	-1.67	-0.58

foreach *step of top-down procedure* **do**
  foreach *candidate split* c **do**
    $T_c \leftarrow$ add split c to T;
    if $Mode = Plain$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_r(p)$ for $p : leaf_r(p) = leaf_r(i))$ for $i = 1..n$;
    if $Mode = Ordered$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_{r,\sigma_r(i)-1}(p)$ for $p : leaf_r(p) = leaf_r(i), \sigma_r(p) < \sigma_r(i))$ for $i = 1..n$;
    $loss(T_c) \leftarrow \cos(\Delta, G)$
  $T \leftarrow \arg\min_{T_c}(loss(T_c))$

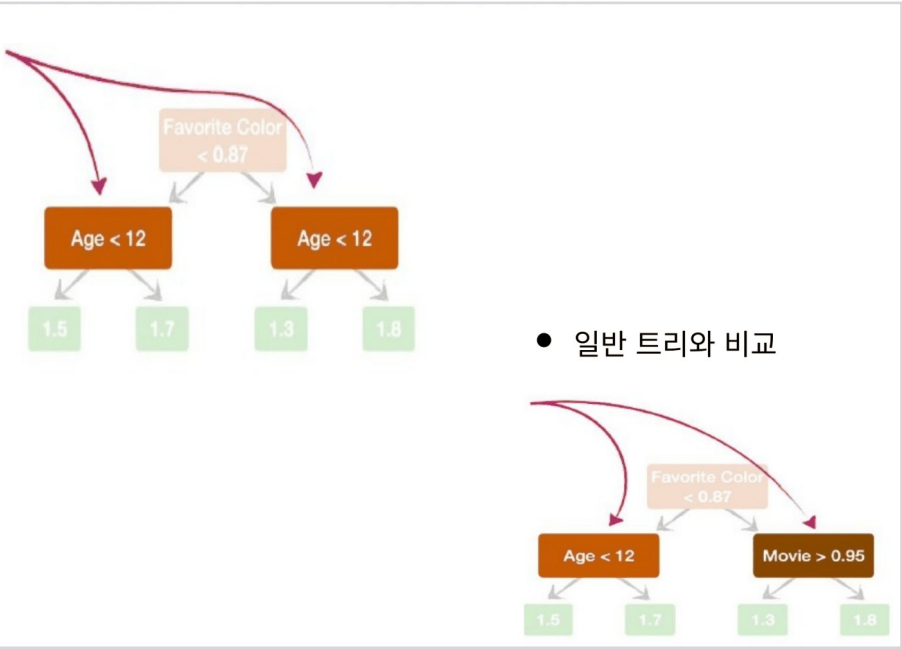
$$loss(T_c) = cos(\Delta, G) = 0.3575$$

(TS < 0.99)

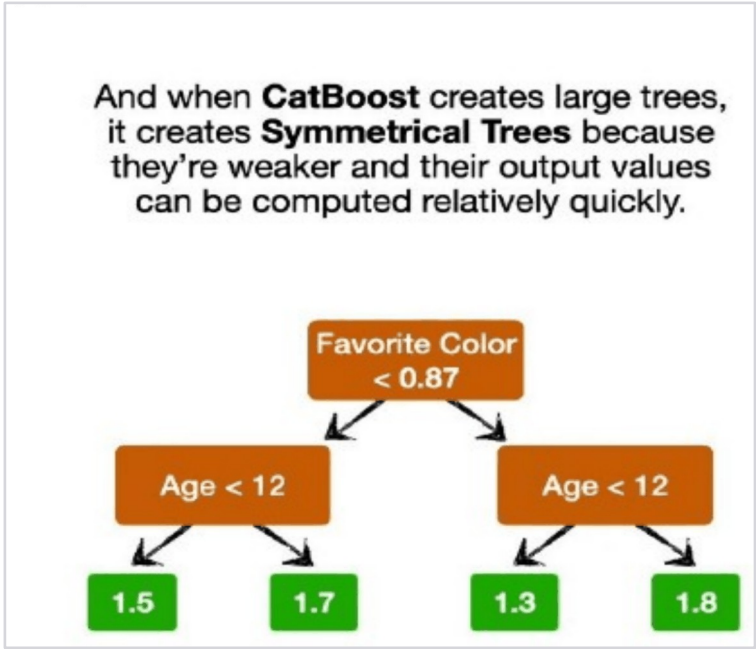
CatBoost 트리는 같은 깊이에서 같은 분할을 쓴다

대칭(oblivious) 트리 — 약한 학습기이지만 계산이 빠르다

일반 트리는 가지마다 다른 질문을 할 수 있다



대칭 트리는 깊이마다 질문이 하나다



질문 순서가 고정돼 있어 벡터 연산 한 번으로 리프를 찾고, 과적합도 줄어든다.

정보가 같은 변수는 하나로 묶는다

순서 특성 조합(Ordered Feature Combination)

읽는 법

- 이 예에서는 국가만 알면 머리색이 정해진다.
- 정보 이득이 같은 두 변수는 하나로 묶는다.
- 전처리에서 변수를 고르는 부담이 줄어든다.

country	hair color	class_label
India	black	1
India	black	1
India	black	1
india	black	1
russia	white	0
russia	white	0
russia	white	0
russia	white	0

CatBoost는 범주형이 많을 때 가장 강하다

기타 특징과 한계 — 원본 정리를 CatBoost 1.2.10 기준으로 보강

항목	내용
원-핫 인코딩	카디널리티가 one_hot_max_size 이하인 범주는 타깃 인코딩 대신 원-핫으로 처리한다
기본 파라미터	기본값이 잘 맞춰져 있어 튜닝으로 얻는 차이가 상대적으로 작다
먼저 볼 파라미터	learning_rate, depth, l2_leaf_reg, random_strength 순으로 조정한다
한계	수치형 변수가 대부분이면 LightGBM보다 학습이 느리다
실행 결과	이 강의의 합성 데이터 실험에서 학습 시간이 다른 두 엔진의 약 8~10배였다

04

PART 4 · 구현

클로드 코드로 구현한다

큰 그림과 기준은 사람이 정하고, 코딩은 클로드 코드에게 맡긴다

세 라이브러리 모두 2026년에 새 버전이 나왔다

클로드 코드가 옛 예제를 따라 쓰지 않도록 버전을 프롬프트에 적는다

3.4.1
XGBoost · 2026년 8월

4.7.0
LightGBM · 2026년 7월

1.2.10
CatBoost · 2026년 2월

라이브러리	알아 둘 변화
XGBoost	3.0에서 feval, 3.1에서 gpu_hist와 gpu_id가 없어졌다. GPU는 device로 고른다
LightGBM	4.0에서 early_stopping_rounds와 verbose_eval 인자가 없어지고 콜백으로 바뀌었다
CatBoost	1.2 계열에서 API가 안정적이다. 원본 코드를 거의 그대로 쓸 수 있다

설치도 클로드 코드에게 맡긴다. 사람은 프롬프트에 버전만 적는다.

구현의 큰 그림은 다섯 단계다

사람이 정하는 것은 단계와 기준이고, 코드는 클로드 코드가 쓴다

① 환경과 데이터

버전을 고정해 설치하고 데이터를 한 번만 나눈다

학습 80%, 검증 20%

② 세 엔진 학습

같은 분할에서 조기 종료로 학습한다

범주형은 내장 기능

③ 비교

RMSE와 학습 시간을 표와 차트로 본다

검증 곡선 포함

④ 튜닝

가장 나은 엔진을 Optuna로 다듬는다

트리 수는 조기 종료로

⑤ 해석

SHAP으로 변수의 영향을 확인한다

beeswarm과 waterfall

프롬프트 ①은 ①~③ 단계를, 프롬프트 ②는 ④~⑤ 단계를 맡는다.

모든 단계는 같은 순서로 클로드 코드에게 시킨다

Claude Code 데스크톱 앱(Code 탭) 기준 — 어디, 입력, 출력, 피드백

어디

Code 탭

작업 폴더를 열고 입력창에 커서를 둔다

예: gbm-lab 폴더

입력

프롬프트

앰버 박스의 프롬프트를 그대로 붙여 넣는다

버전과 기준을 적는다

출력

코드와 결과

클로드 코드가 코드를 쓰고 실행해 결과를 보여 준다

파일, 표, 차트

피드백

검토

주의할 점에 어긋난 곳을 짚어 고치게 한다

점검 프롬프트 사용

수강생은 코드를 직접 쓰지 않는다. 기준을 주고 결과를 검토한다.

프롬프트 하나로 세 엔진의 비교 실험을 만든다

단계 ①~③ — 환경 설치, 데이터 생성, 같은 분할에서 학습과 비교

[입력] 프롬프트 · Claude Code (Code 탭)

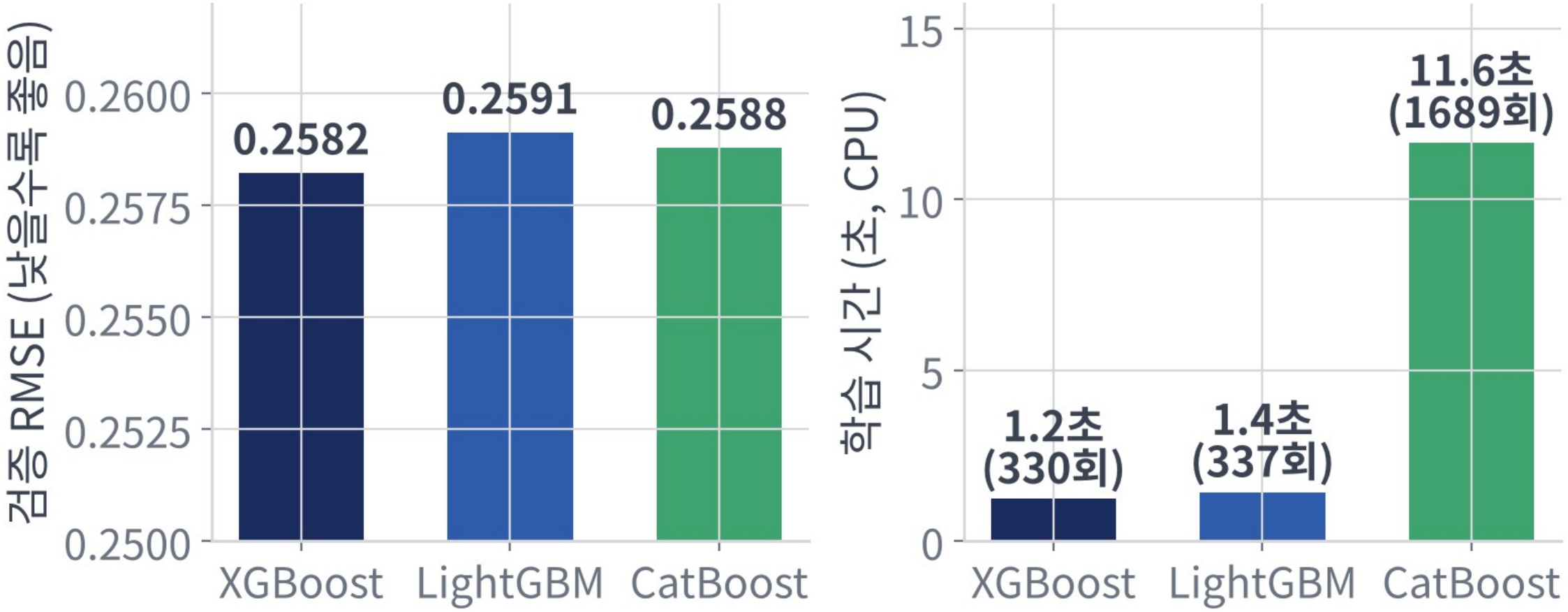
```
gbm-lab 폴더에서 다음을 해 줘.
1. 가상환경을 만들고 xgboost 3.4, lightgbm 4.7,
   catboost 1.2.10을 설치해 requirements.txt로 고정해.
2. 주택가격 합성 데이터 3만 행을 만들어 줘.
   수치형 6개, 범주형 2개(region 60개 수준, house_type).
3. 학습 80%, 검증 20%로 한 번만 나누고
   세 엔진이 같은 분할을 쓰게 해.
4. 세 엔진을 조기 종료(100회)로 학습하고 검증 RMSE,
   학습 시간, 최적 반복 수를 results.csv와
   막대 차트로 저장해. 검증 곡선도 그려 줘.
5. 범주형은 각 엔진의 내장 기능으로 처리하고
   원-핫 인코딩은 쓰지 마.
6. 2026년 기준 API로 쓰고, 경고가 나면 알려 줘.
```

[출력] 결과 · 클로드 코드가 만든 파일

```
requirements.txt
  버전 고정
data.py
  합성 데이터 생성
train_compare.py
  세 엔진 학습과 비교
results.csv
  RMSE, 시간, 반복 수
bench.png, curves.png
  비교 차트와 검증 곡선
```

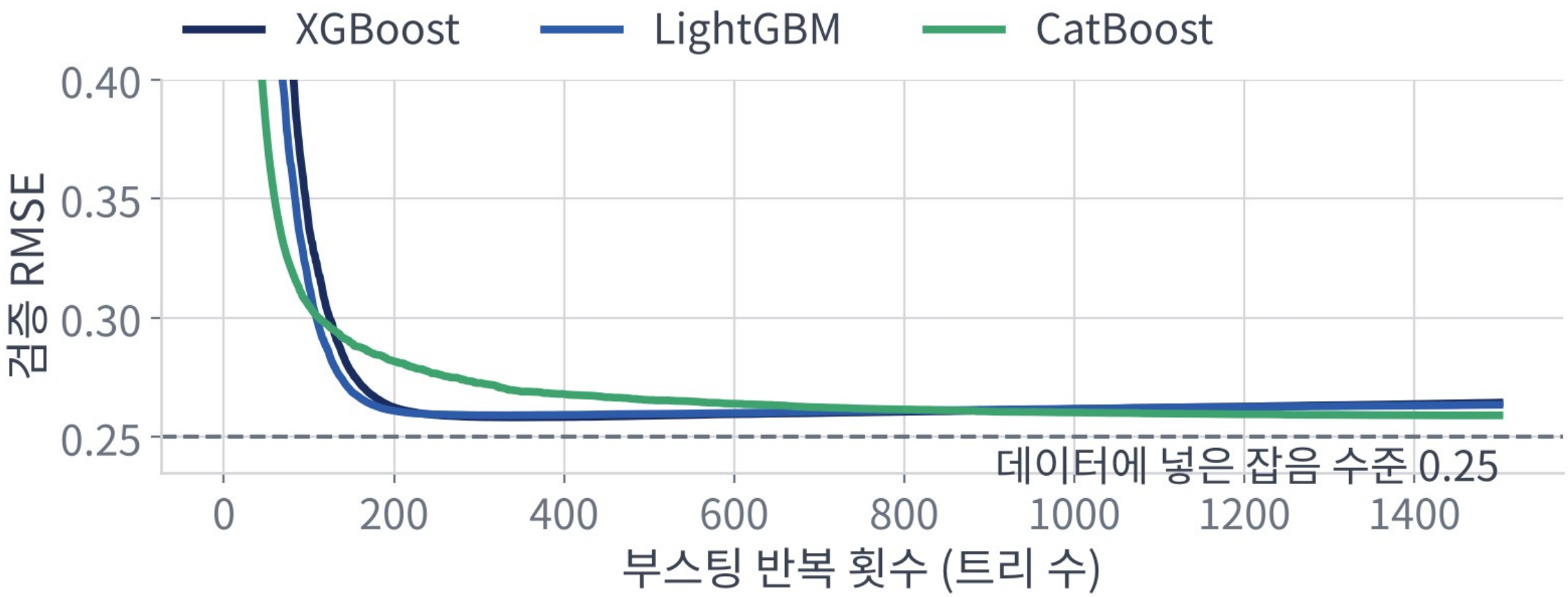
같은 데이터에서 세 엔진의 정확도 차이는 작았다

프롬프트 ①을 실행한 결과 — 합성 데이터 3만 행, CPU (※ 합성 데이터)



조기 종료가 과적합 직전의 트리 수를 찾아 준다

검증 RMSE 곡선 — XGBoost와 LightGBM은 약 330회 뒤로 오차가 다시 오른다



튜닝과 해석도 기준만 정해 맡긴다

단계 ④~⑤ — Optuna 튜닝과 SHAP 해석

[입력] 프롬프트 · Claude Code (Code 탭)

```
results.csv에서 가장 좋았던 엔진을 Optuna로 튜닝해 줘.
- 시도 50회, 목적은 검증 RMSE 최소화
- 탐색 대상: 학습률(로그 척도), 트리 크기,
  리프 최소 크기, 열 샘플링 비율, L2 규제
- 트리 수는 탐색하지 말고 조기 종료로 정해
- 튜닝 전후 RMSE를 표로 비교해 줘

그다음 최종 모델로 SHAP 분석을 해 줘.
- beeswarm 그림과 관측치 하나의 waterfall 그림
- 상위 5개 변수의 영향 방향을 한 줄씩 설명해 줘
```

검토할 것

- 트리 수를 조기 종료로 정했는가?
- 검증 구간으로 튜닝했다면 최종 성능은 따로 떼어 둔 테스트 구간으로 확인했는가?
- SHAP 설명이 데이터를 만든 방식과 맞는가?

클로드 코드가 옛 코드를 쓰면 이 표로 바로잡는다

원본 강의 코드와 2026년 코드 대응표 — 각 라이브러리 릴리스 노트 기준

옛 코드	2026년 코드	이유
'objective': 'reg:linear'	'reg:squarederror'	폐기된 옛 이름이다
'silent': 1	'verbosity': 0	silent는 더 이상 쓰지 않는다
feval=rmspe_xg	custom_metric=rmspe	3.0에서 feval이 제거됐다
tree_method='gpu_hist'	device='cuda'	3.1에서 GPU 전용 이름이 제거됐다
lgb.train(..., verbose_eval=10, early_stopping_rounds=100)	callbacks=[lgb.early_stopping(100), lgb.log_evaluation(10)]	4.0에서 인자가 콜백으로 바뀌었다
XGBClassifier().fit(..., early_stopping_rounds=)	XGBClassifier(early_stopping_rounds=)	생성자 인자로 옮겨졌다

같은 역할의 파라미터가 엔진마다 이름이 다르다

핵심 하이퍼파라미터 대응표 — 사이킷런 래퍼 이름 기준, 괄호는 별칭이나 네이티브 이름

역할	XGBoost	LightGBM	CatBoost
학습률	learning_rate (eta)	learning_rate	learning_rate (eta)
트리 수	n_estimators	n_estimators (num_iterations)	iterations
트리 크기	max_depth (기본 6)	num_leaves (기본 31)	depth (기본 6)
리프 최소 크기	min_child_weight	min_child_samples	min_data_in_leaf
행 샘플링	subsample	subsample (bagging_fraction)	subsample
열 샘플링	colsample_bytree	colsample_bytree (feature_fraction)	rsm
L2 규제	reg_lambda	reg_lambda (lambda_l2)	l2_leaf_reg
분할 최소 이득	gamma	min_split_gain	해당 없음

별칭을 함께 쓰면 한쪽이 조용히 무시된다

직접 실행해 확인한 동작 — XGBoost 3.4.1, LightGBM 4.7.0, CatBoost 1.2.10

상황	실제 동작	대책
XGBoost에 eta와 learning_rate를 함께 준다	경고 없이 learning_rate 값이 쓰였다	둘 중 하나만 쓴다
LightGBM에 bagging_fraction과 subsample을 함께 준다	경고를 한 줄 내고 subsample을 무시했다	한 가지 이름 체계로 통일한다
LightGBM params에 n_estimators를 넣고 num_boost_round도 준다	params 쪽이 이겼다 (5회 대신 20회 학습)	트리 수는 한 곳에서만 정한다
CatBoost에 learning_rate와 eta를 함께 준다	학습을 시작할 때 오류가 났다	하나만 쓴다

프롬프트에 '파라미터는 한 가지 이름으로만 써 줘'라고 적어 둔다.

이름이 같아도 뜻이 다른 파라미터가 있다

엔진을 바꿔 가며 같은 값을 넣을 때 생기는 함정

파라미터	주의할 점
min_child_weight	XGBoost와 LightGBM 모두 관측치 수가 아니라 헤시안 합이다. LightGBM에서 관측치 수 기준은 min_child_samples(기본 20)다
max_depth	XGBoost는 기본 6, LightGBM은 기본 −1(제한 없음)이다. LightGBM의 트리 크기는 num_leaves가 정한다
min_data_in_leaf	CatBoost 문서상 Depthwise와 Lossguide 성장 방식에서만 쓰인다. 기본 대칭 트리에 넣어도 오류 없이 넘어간다
난수 시드	네이티브 이름은 XGBoost와 LightGBM이 seed, CatBoost가 random_seed다. 사이킷런 래퍼는 모두 random_state를 받는다

실행 단계에서 자주 걸리는 함정은 네 가지다

이 강의의 실습 코드를 실행하며 실제로 확인한 동작

상황	주의할 점
CatBoost 범주형 열의 결측	NaN이 있으면 오류가 난다. 'missing' 같은 문자열로 먼저 채운다
XGBoost 조기 종료 뒤 예측	사이킷런 predict는 최적 반복까지만 쓰지만, Booster.predict는 iteration_range를 주지 않으면 모든 트리를 쓴다
LightGBM 튜닝	min_child_samples를 바꿔 가며 탐색하면 오류가 난다. Dataset에 feature_pre_filter=False를 준다
LightGBM 배깅	bagging_fraction은 bagging_freq가 0보다 커야 작동한다

피드백

마지막에 클로드 코드에게 스스로 점검하게 한다

피드백 단계 — 주의 ①~⑤를 체크리스트로 바꿔 그대로 붙여 넣는다

[입력] 프롬프트 · Claude Code (Code 탭)

```
지금까지 만든 코드를 아래 기준으로 점검하고,
문제가 있으면 고친 뒤 무엇을 고쳤는지 알려 줘.
1. 설치된 버전이 requirements.txt와 맞는가
2. 폐기된 이름(reg:linear, silent, feval, gpu_hist,
   verbose_eval, early_stopping_rounds 인자)이 없는가
3. 같은 파라미터를 별칭으로 두 번 주지 않았는가
4. 세 엔진이 같은 학습·검증 분할을 썼는가
5. 예측에 최적 반복 수까지만 썼는가
6. 범주형 열의 결측을 어떻게 처리했는가
7. 실행 중 나온 경고를 모두 보여 줘
```

보고에서 볼 것

- 고친 곳마다 이유가 적혀 있는가?
- 경고가 남아 있으면 원인을 다시 묻는다.
- 숫자가 바뀌었으면 비교표를 다시 받는다.

엔진은 데이터 구조와 속도 요구로 고른다

실무의 출발점 — 최종 선택은 같은 검증 구간에서 비교해 정한다



작은 표 데이터에서는 파운데이션 모델이 새 경쟁자다

TabPFN-2.5 (Prior Labs, 2025년 11월) — 학습 없이 문맥 학습으로 예측한다

보고된 성과

- 5만 행, 2천 변수 이하 데이터를 대상으로 설계됐다.
- 기본 설정 XGBoost와 비교한 분류 승률은 1만 행 이하에서 100%, 10만 행 이하에서 87%였다.

부스팅이 여전히 강한 곳

- 행이 수십만 개를 넘는 데이터에서 빠르다.
- 논문의 비교 대상은 기본 설정 모델이라, 튜닝한 모델과의 비교는 따로 확인해야 한다.
- 배포가 가볍고 해석 도구가 성숙했다.

대용량 표 데이터의 기본값은 여전히 GBDT다. 작은 데이터에서는 TabPFN도 함께 비교한다.

토론으로 원리를 실무 판단에 연결한다

각 질문에 이 강의의 공식과 그림을 근거로 답한다

Q1	λ와 γ를 함께 키우면 트리 모양과 리프 출력은 각각 어떻게 달라지는가?	**XGBoost 규제화**
Q2	결측이 많은 변수에서 희소 인지 분할이 대치보다 나은 경우와 위험한 경우는?	**결측값 처리**
Q3	num_leaves를 크게 늘리면 LightGBM에서 어떤 위험이 커지고, 무엇으로 막는가?	**리프 중심 성장**
Q4	카디널리티가 높은 범주형 변수를 타깃 인코딩할 때 누수를 어떻게 막는가?	**Ordered TS**

세 엔진은 같은 뼈대 위의 서로 다른 최적화다

이 강의의 결론

1

XGBoost 트리는 공식 네 개로 만들어진다

잔차의 합과 개수, 그리고 λ와 γ가 분할과 가지치기와 출력값을 정한다.

2

LightGBM은 계산할 데이터를 줄여 속도를 얻는다

리프 중심 성장으로 손실을 빨리 줄이고, GOSS와 EFB로 행과 열을 줄인다.

3

CatBoost는 순서를 지켜 누수를 막는다

앞선 행만으로 인코딩과 잔차를 계산하고, 대칭 트리로 빠르게 예측한다.

구현은 클로드 코드에게 맡기고, 사람은 버전과 별칭 같은 주의 사항을 검토한다.

A

부록

부록 — 논문 알고리즘과 CatBoost 손계산

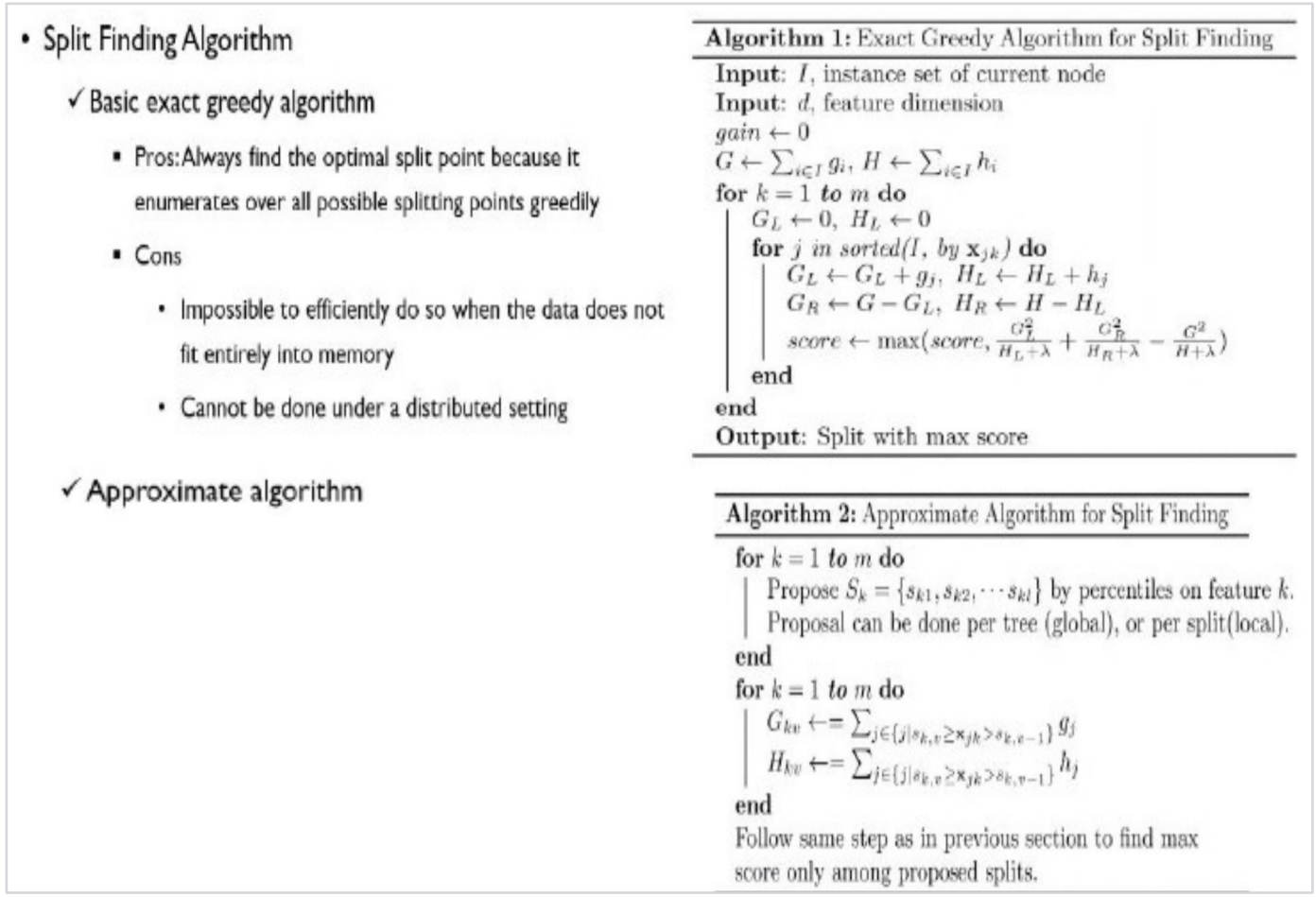
XGBoost 논문의 알고리즘과 실험, CatBoost 트리를 StatQuest 예제로 끝까지 따라가기

논문은 정확한 탐욕과 근사 알고리즘을 구분한다

XGBoost 논문 Algorithm 1과 2 (Chen & Guestrin, 2016)

읽는 법

- 정확한 탐욕은 모든 분할점을 정렬해 훑는다.
- 데이터가 메모리에 다 들어가지 않거나 분산 환경이면 쓸 수 없다.
- 근사 알고리즘은 분위수 후보만 훑는다.



- Split Finding Algorithm
  - ✓ Basic exact greedy algorithm
    - Pros: Always find the optimal split point because it enumerates over all possible splitting points greedily
    - Cons
      - Impossible to efficiently do so when the data does not fit entirely into memory
      - Cannot be done under a distributed setting
  - ✓ Approximate algorithm

Algorithm 1: Exact Greedy Algorithm for Split Finding

Input: I, instance set of current node
Input: d, feature dimension
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I} g_i$, $H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ *to* m **do**
  $G_L \leftarrow 0$, $H_L \leftarrow 0$
  for j *in sorted*$(I,$ *by* $\mathbf{x}_{jk})$ **do**
    $G_L \leftarrow G_L + g_j$, $H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L$, $H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split with max score

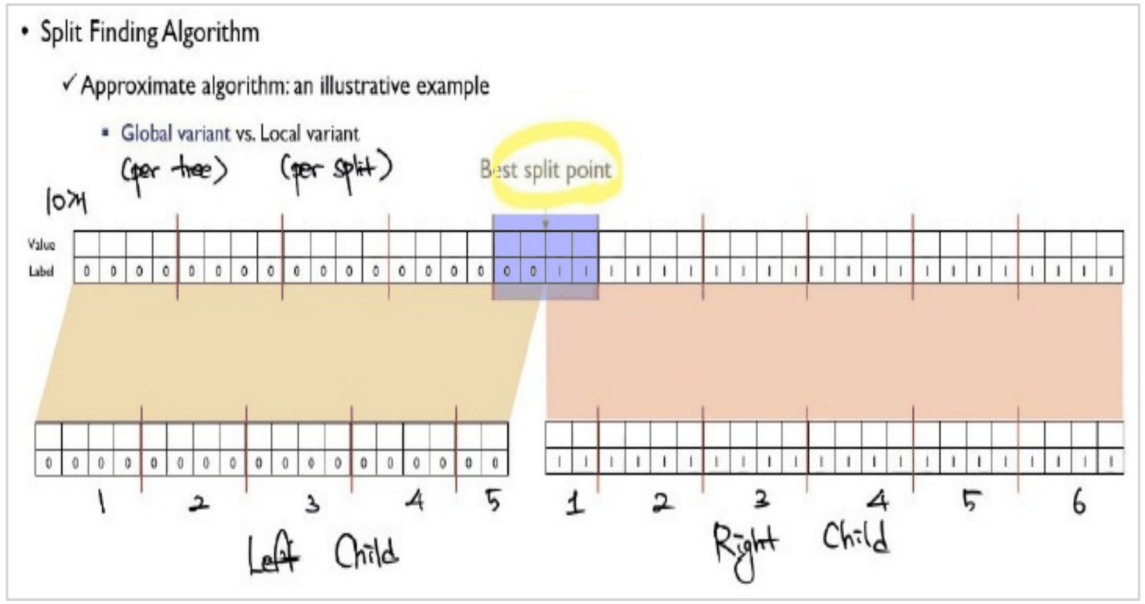
Algorithm 2: Approximate Algorithm for Split Finding

for $k = 1$ *to* m **do**
  Propose $S_k = \{s_{k1}, s_{k2}, \cdots s_{kl}\}$ by percentiles on feature k.
  Proposal can be done per tree (global), or per split(local).
end
for $k = 1$ *to* m **do**
  $G_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} g_j$
  $H_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} h_j$
end
Follow same step as in previous section to find max score only among proposed splits.

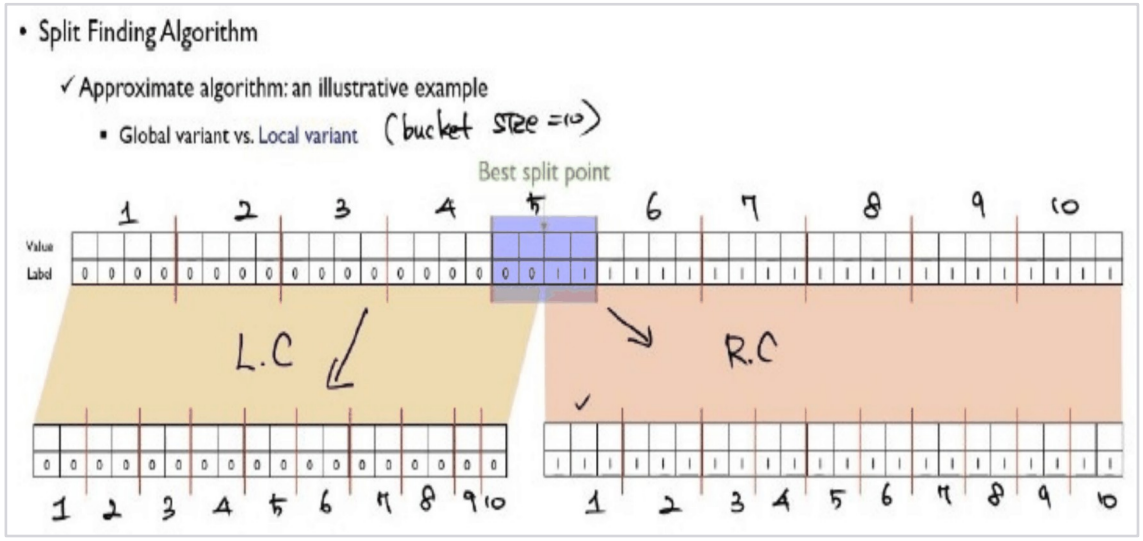
후보 분위수는 한 번 정하거나 분할마다 정한다

global 제안과 local 제안 — 원본의 손그림 설명

global — 트리를 만들기 전에 후보를 한 번 정한다



local — 분할할 때마다 후보를 다시 정한다

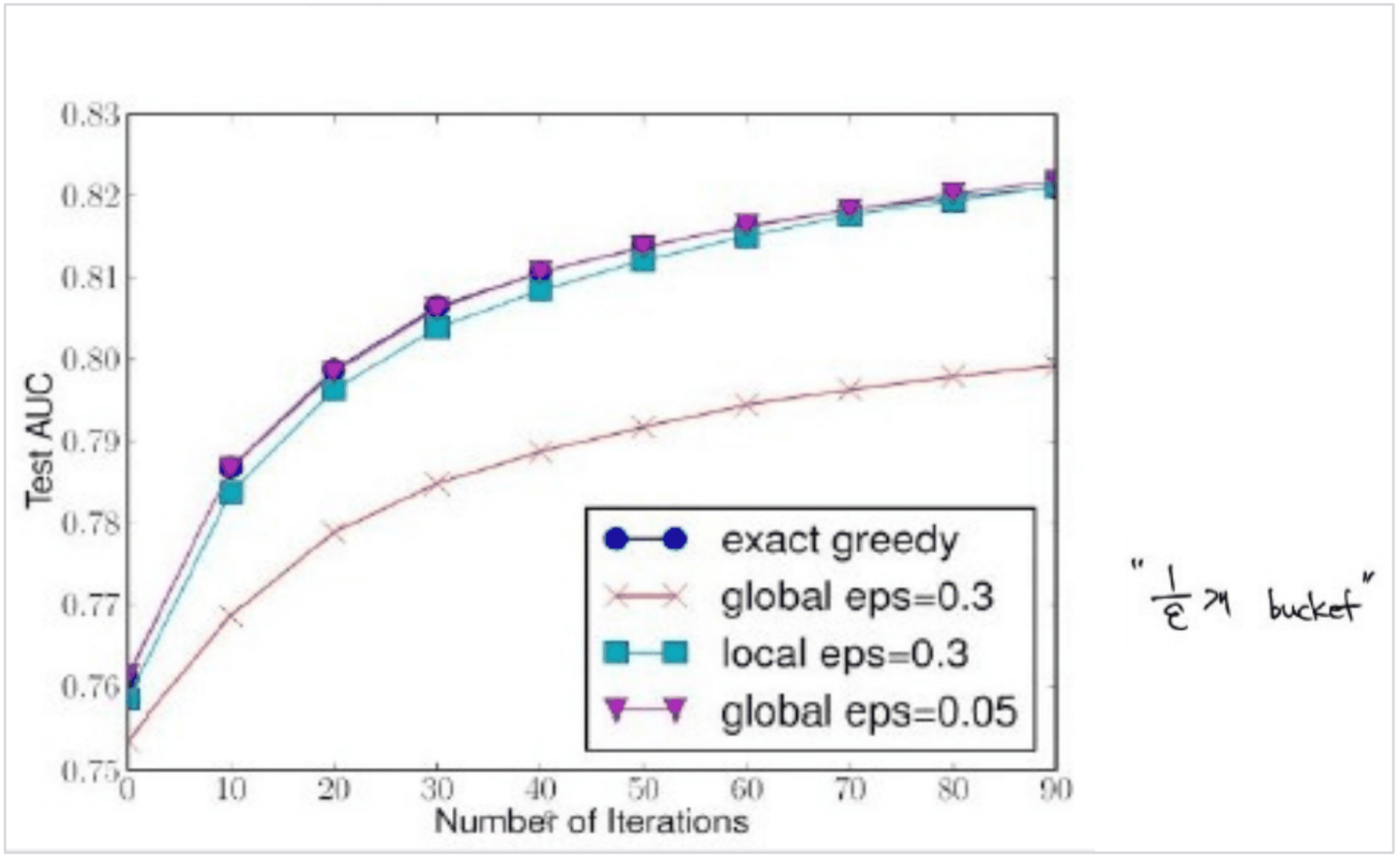


global 제안은 후보가 촘촘해야 local에 가까워진다

논문 Figure 3 — eps는 분위수 간격으로, 작을수록 후보가 많다

읽는 법

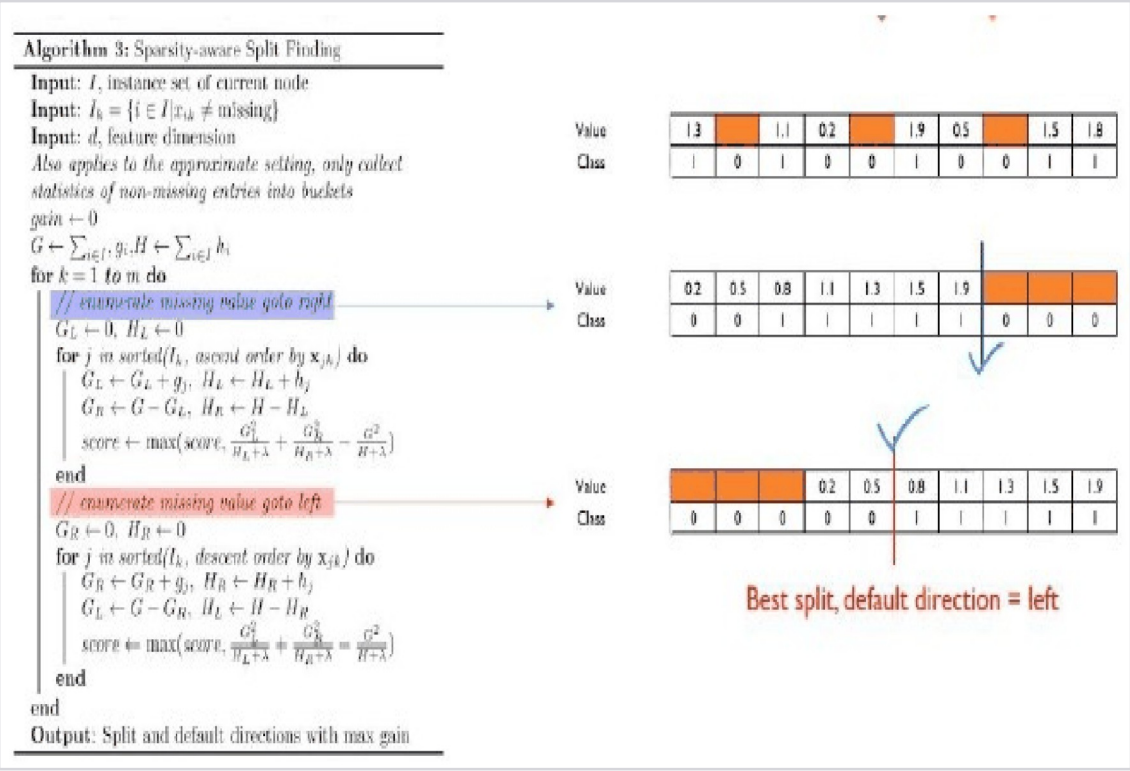
- global eps 0.3은 정확도가 크게 떨어진다.
- global eps 0.05면 local eps 0.3과 비슷해진다.
- 후보가 많을수록 정확하지만 느리다.



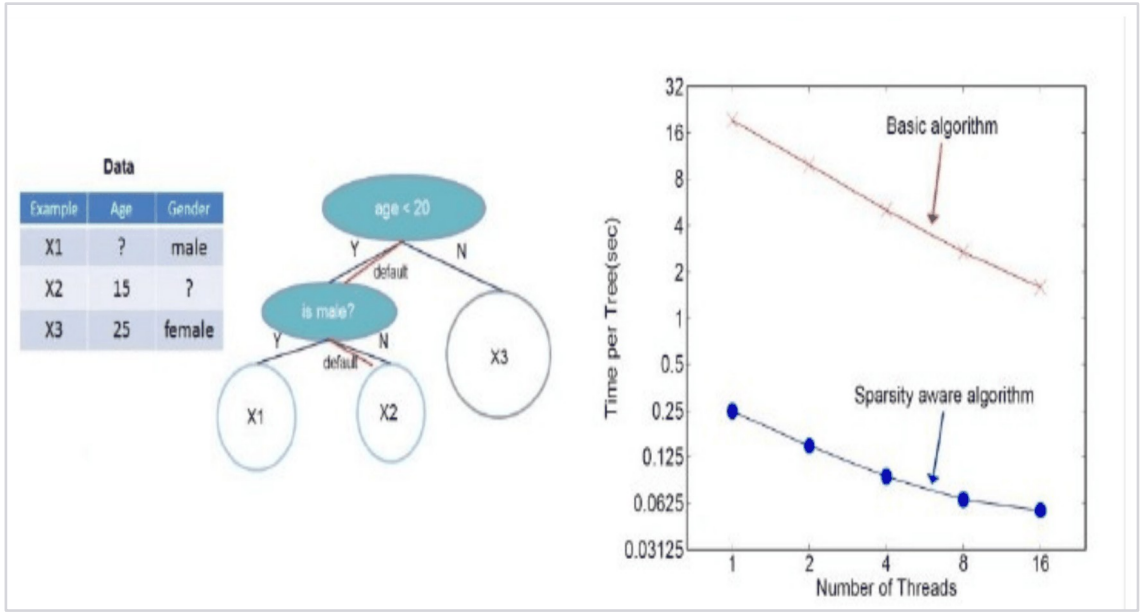
희소 인지 알고리즘은 결측이 아닌 값만 훑는다

논문 Algorithm 3과 Figure 5 — 희소 데이터에서 기본 알고리즘보다 50배가량 빠르다

결측값을 오른쪽과 왼쪽에 보내 두 번 훑는다



스레드 수와 관계없이 기본 알고리즘보다 빠르다

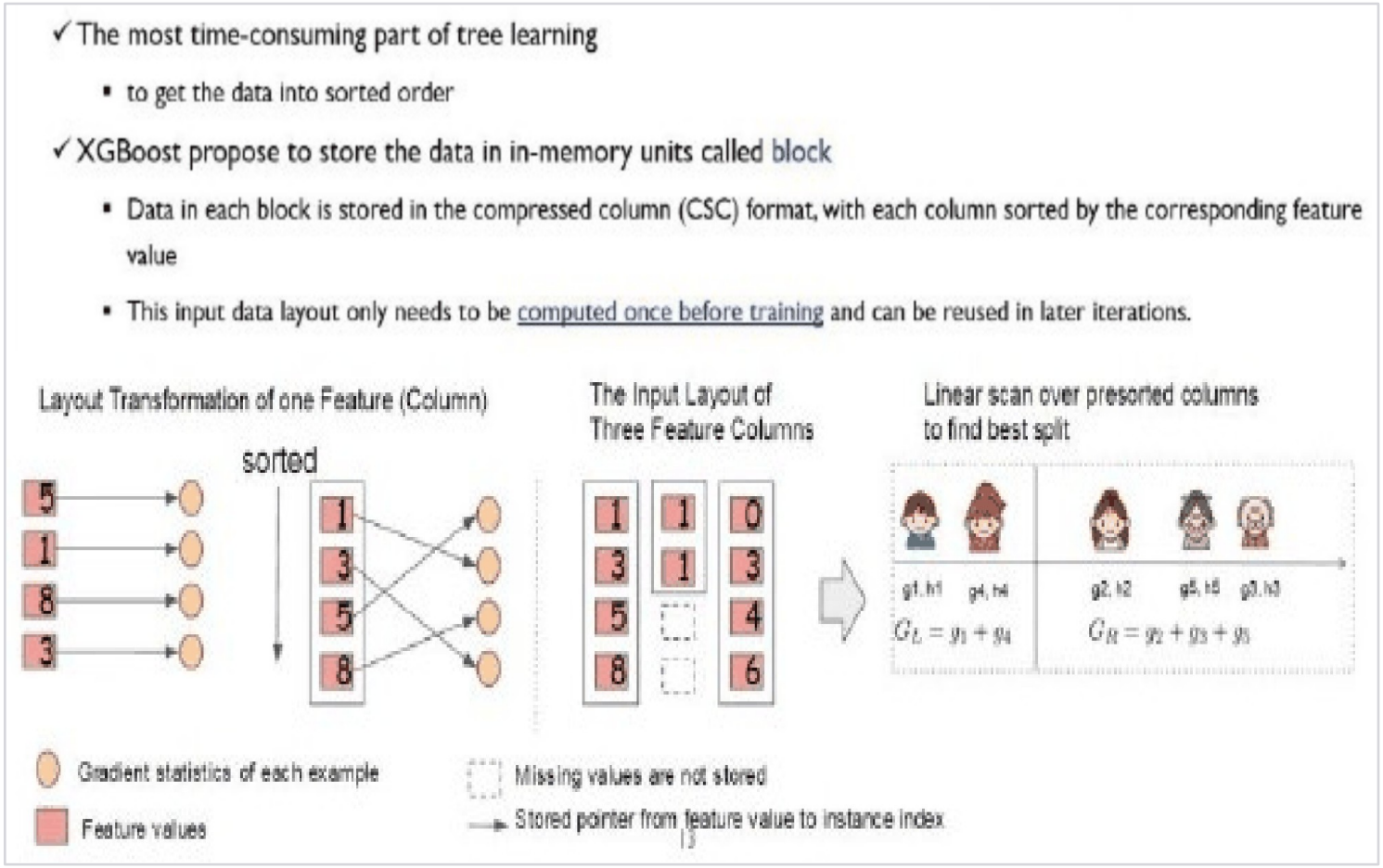


블록 구조로 정렬을 한 번만 한다

논문 System Design — 열 단위 압축 블록(CSC)

읽는 법

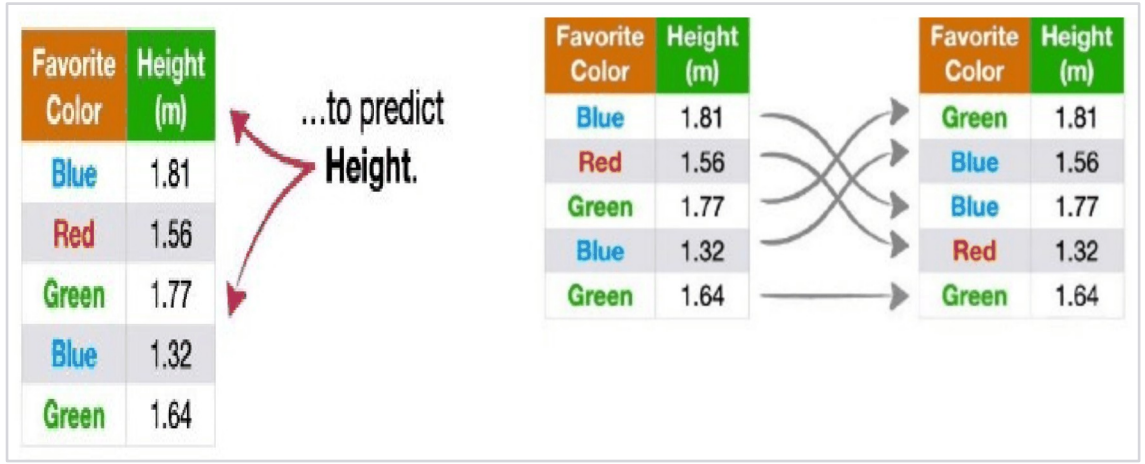
- 트리 학습에서 가장 비싼 일은 값을 정렬하는 것이다.
- 열마다 정렬해 블록에 저장하고 반복마다 재사용한다.
- 블록은 압축해 디스크에서 읽고, 다른 스레드가 압축을 푼다.



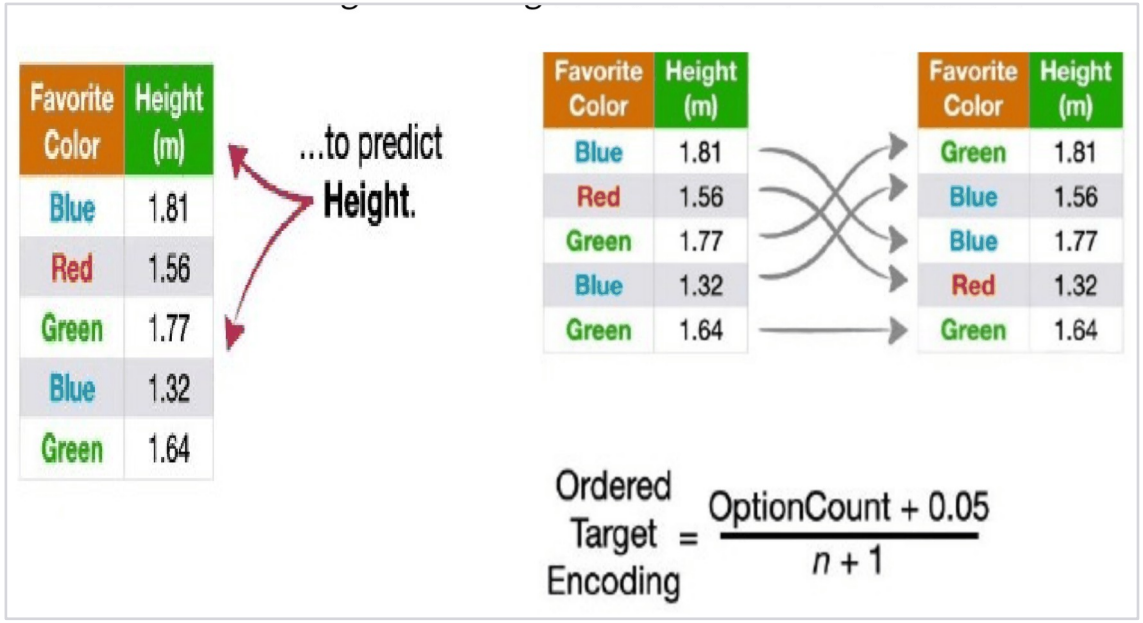
먼저 행을 섞고 범주형 열을 순서대로 인코딩한다

StatQuest 예 — Favorite Color로 키(Height)를 예측하는 CatBoost 트리

훈련 데이터의 행 전체를 무작위로 섞는다



범주형 열에 Ordered Target Encoding을 적용한다



회귀 문제라 키를 두 구간(Bin)으로 나눈 뒤, 그 구간 번호로 인코딩한다.

예측을 0으로 시작해 잔차와 분할 후보를 만든다

잔차 = 관측값 − 예측값, 분할 후보는 정렬한 인코딩 값의 사이다

처음에는 예측이 0이라 잔차가 키 그대로다

Favorite Color	Height (m)	Predictions	Residuals
0.05	1.81	0	
0.05	1.56	0	
0.025	1.77	0	
0.05	1.32	0	
0.525	1.64	0	

Residual = Observed - Predicted

Favorite Color	Height (m)	Predictions	Residuals
0.05	1.81	0	1.81

Residual = Observed - Predicted

Residual = 1.81 - 0 = 1.81

Favorite Color	Height (m)	Predictions	Residuals
0.05	1.81	0	1.81
0.05	1.56	0	1.56
0.025	1.77	0	1.77
0.05	1.32	0	1.32
0.525	1.64	0	1.64

후보 경계는 0.04와 0.29다

Favorite Color
0.05
0.05
0.025
0.05
0.525

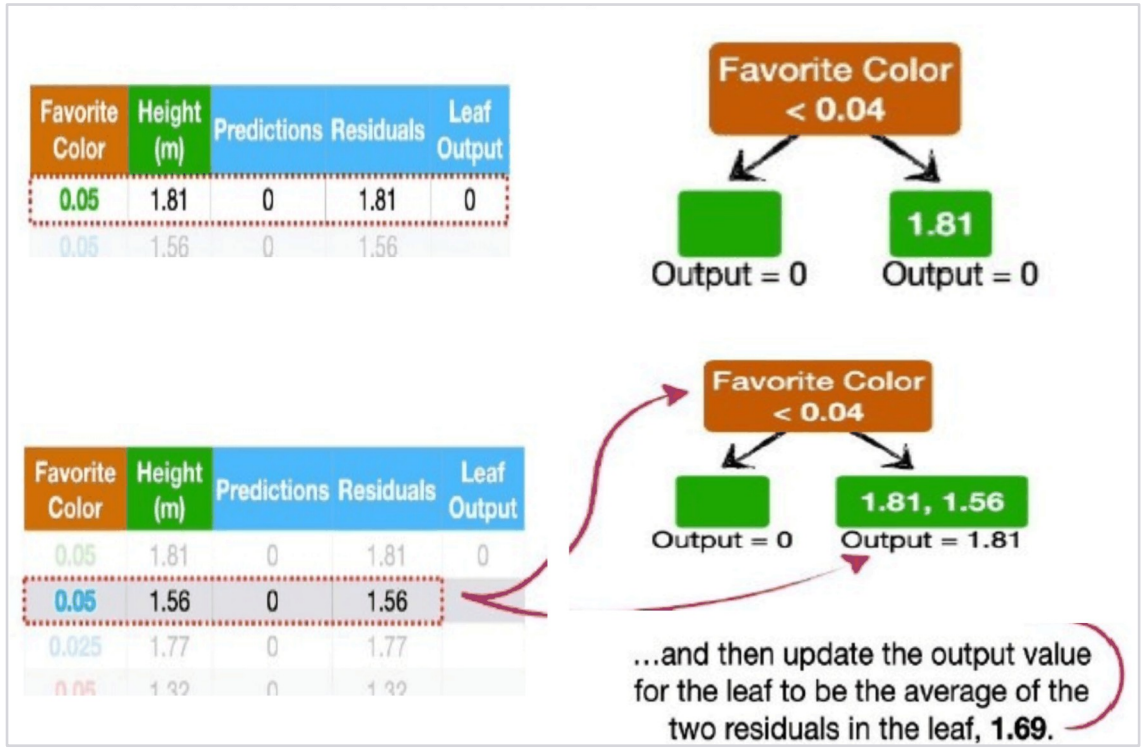
Sorted

Favorite Color	
0.025	0.04
0.05	
0.05	
0.05	0.29
0.525	

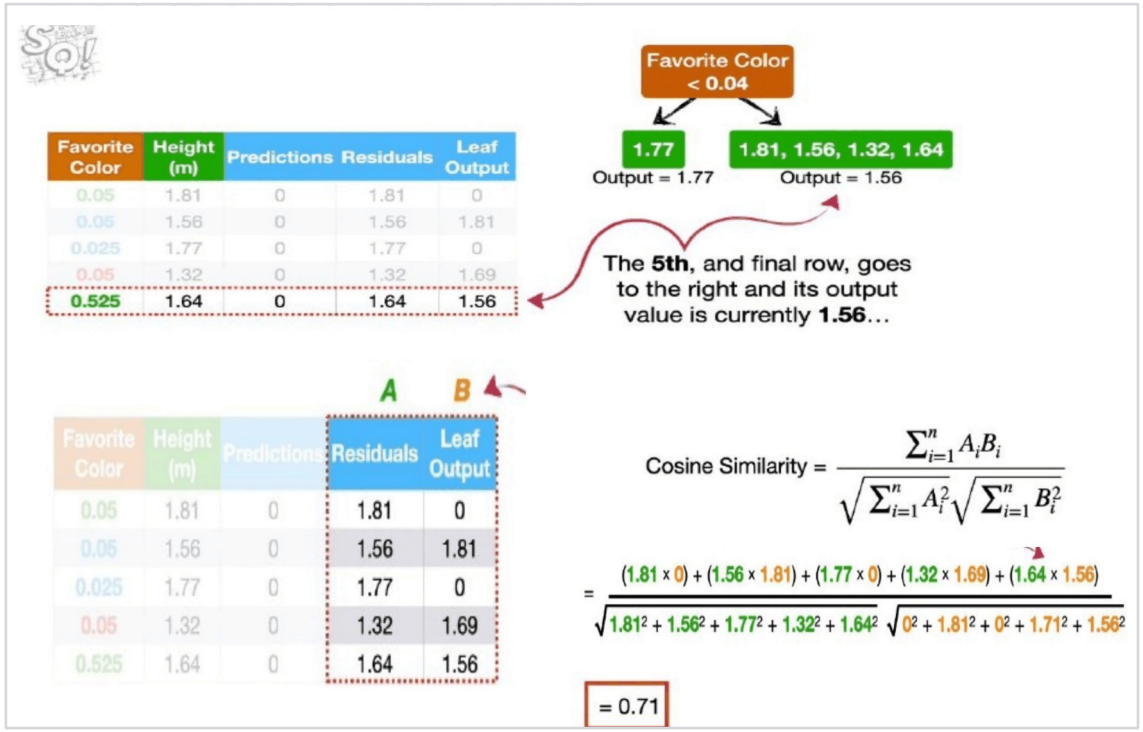
리프 출력은 앞선 행의 잔차만으로 계산한다

한 행씩 내려가며 리프 출력을 기록하고 코사인 유사도를 구한다

자기 행의 잔차는 그 행의 리프 출력에 넣지 않는다



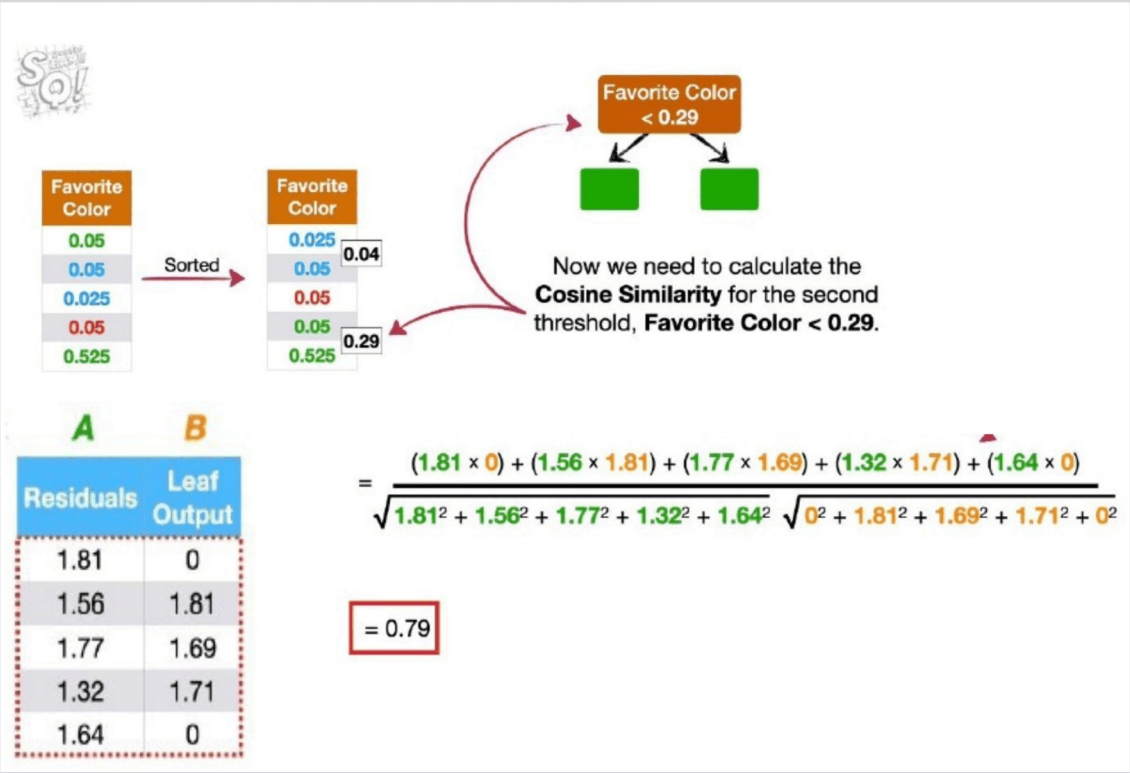
경계 0.04의 코사인 유사도는 0.71이다



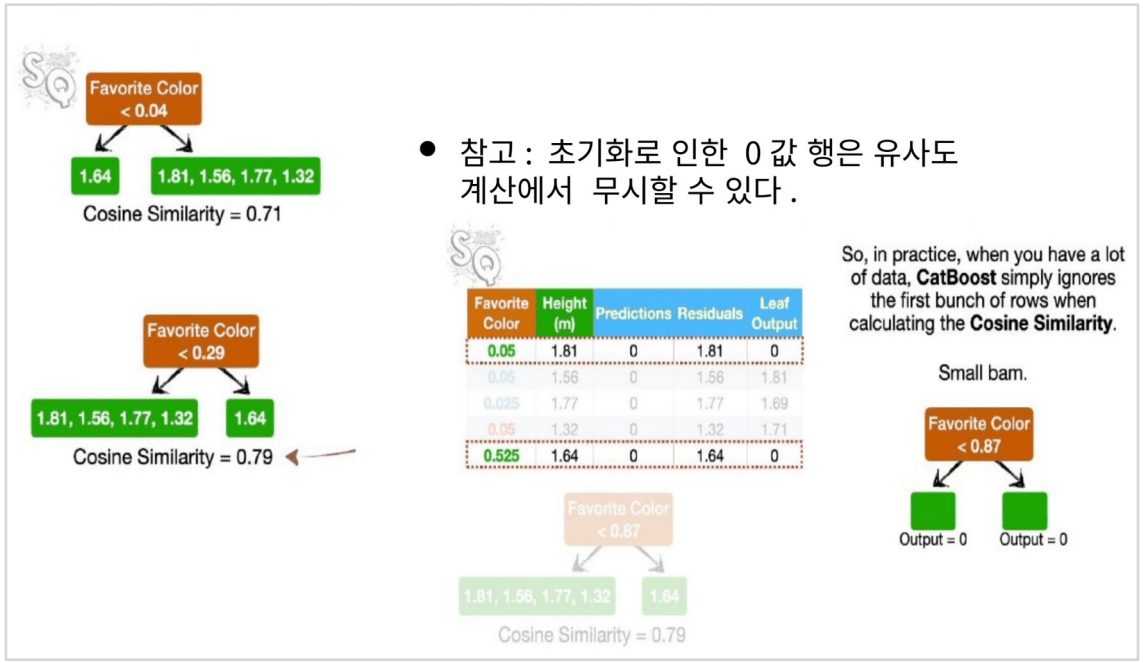
유사도가 더 큰 경계 0.29를 고른다

같은 계산을 경계 0.29에도 적용한다

경계 0.29의 코사인 유사도는 0.79다



0.79가 0.71보다 커서 0.29를 고른다



- 참고 : 초기화로 인한 0 값 행은 유사도 계산에서 무시할 수 있다 .

예측을 갱신하고 새 잔차를 구한다

새 예측은 기존 예측에 학습률 0.1 × 리프 출력을 더한 값이다

학습률 0.1로 예측을 갱신한다

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0		
0.05	1.56	0.18		
0.025	1.77	0.17		
0.05	1.32	0.17		
0.525	1.64	0		0

Learning Rate = 0.1

New Prediction = Prediction + (Learning Rate × Leaf Output)

New Prediction = 0 + (0.1 × 0)

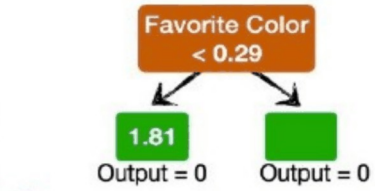
New Prediction = 0.0

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0		
0.05	1.56	0.18		
0.025	1.77	0.17		
0.05	1.32	0.17		
0.525	1.64	0		

The **Main Idea** of how **CatBoost** calculates the output values for trees is that, just like for **Ordered Target Encoding**, it treats the data as if it is receiving it sequentially.

잔차는 그 행을 본 적 없는 리프 출력으로 계산된다

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0	1.81	0
0.05	1.56	0.18	1.56	1.81
0.025	1.77	0.17	1.77	0
0.05	1.32	0.17	1.32	1.69
0.525	1.64	0	1.64	1.56



This means that the **Residual** in a row is not part of the calculation of the **Leaf Output** for that same row…

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0	1.81	
0.05	1.56	0.18	1.38	
0.025	1.77	0.17	1.6	
0.05	1.32	0.17	1.15	
0.525	1.64	0		

Residual = Observed - Predicted

다음 트리는 행을 다시 섞어 인코딩부터 새로 한다

트리마다 순서가 달라져 순서 효과가 평균된다

읽는 법

- 인코딩 값을 원래 색 이름으로 되돌린다.
- 행을 다시 섞고 Ordered Target Encoding을 새로 한다.
- 같은 방식으로 분할을 고르고 리프 출력을 갱신한다.

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
Green	1.81	0	1.81	
Blue	1.56	0.18	1.38	
Blue	1.77	0.17	1.6	
Red	1.32	0.17	1.15	
Green	1.64	0	1.64	

- 다음 이전처럼 randomize 하고 , Order target encoding 한다 .

Favorite Color	Height (m)	Bin #	Predictions	Residuals	Leaf Output
Red	1.32	0	0.17	1.15	
Green	1.64	1	0	1.64	
Green	1.81	1	0	1.81	
Blue	1.77	1	0.17	1.6	
Blue	1.56	0	0.18	1.38	

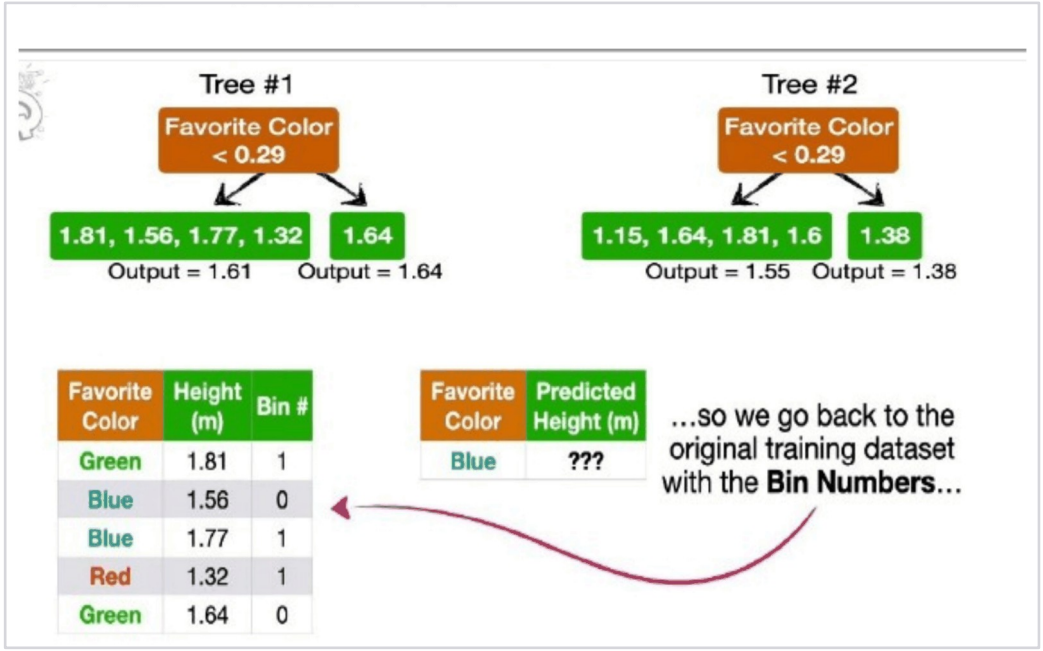


Favorite Color	Height (m)	Bin #	Predictions	Residuals	Leaf Output
0.05	1.32	0	0.17	1.15	
0.05	1.64	1	0	1.64	
0.525	1.81	1	0	1.81	
0.05	1.77	1	0.17	1.6	
0.525	1.56	0	0.18	1.38	

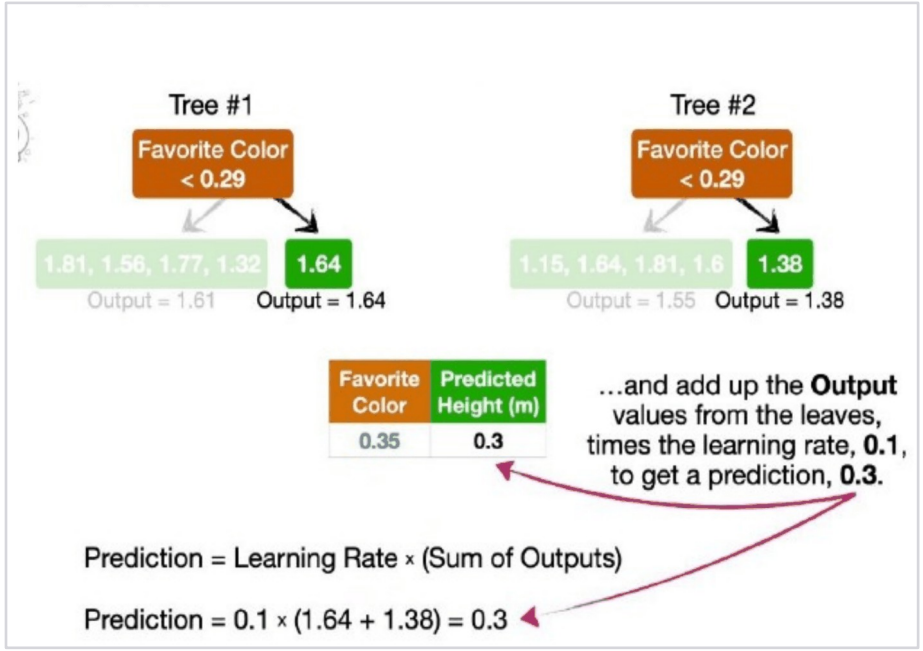
새 관측치는 두 트리를 지나 예측한다

예측 = 학습률 × 리프 출력의 합 = 0.1 × (1.64 + 1.38) = 0.3

트리 두 개를 만들었다



새 관측치 Blue를 0.35로 인코딩해 두 트리에 넣는다



데이터와 변수가 많고 트리가 커지면 예측은 더 좋아진다.